Minor Project Report

on

MULTI-USER CHATROOM APPLICATION

PulseChat

Submitted to:

Dr. Gagandeep Kaur-E12617

Associate Professor

University Institute of Computing

Submitted by:

Asmi Choudhary (25 MCA20325)

Master of Computer Applications (MCA)

Chandigarh University, Mohali

BONAFIDE CERTIFICATE

Certified that this project report "Multi-user Chatroom Application" is the bona fide work of "Asmi Choudhary" who carried out the project work under my/our supervision.

This project report is submitted in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA).

SIGNATURE	SIGNATURE
Dr. Amanpreet Kaur Sandhu	Dr. Gagandeep Kaur-E12617
HEAD OF THE DEPARTMENT	Associate Professor
University Institute of Computing	University Institute of Computing

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Signature and examination details are to be completed by the institution.

Table Of Contents

CHAPTER 1: INTRODUCTION 9-13
CHAPTER 2: BACKGROUND STUDY 14-20
CHAPTER 3: DESIGN AND PROCESS 21-31
CHAPTER 4: RESULT ANALYSIS AND VALIDATION 32-34
CHAPTER 5: CONCLUSION AND FUTURE WORK 35-72
1.1. Need Identification 9
Real-time delivery and data persistence 9
1.1 Need Identification: User needs 9
1.1 Need Identification: Scope 10
1.2. Identification of Problem 10
Problem statement and security considerations 10
1.2 Problem statement and expected outcome 11
1.3. Identification of Tasks 11
Frontend and backend development 11
Database and account management 12
Real-time events and integration 12
Verification and documentation 12
1.4. Timeline 12
Schedule evidence and report structure 12
1.4 Timeline and 1.5 Report organization 13
2.1. Timeline of the Reported Problem 14
Persistent data and session state 14
Browser and server architecture 14
Frontend and backend frameworks 14
History retrieval and duplicate handling 14
Presence and concurrent sessions 15
2.2. Proposed Solution 15
Room and history capabilities 15
Live events and expected benefits 16
User feedback and operational boundaries 16
Local configuration and deployment scope 16
2.3. Bibliometric Analysis 16
WebSocket and STOMP messaging 17
Publish-subscribe design 17
Document storage model 17
Component-based interface design 17
Security considerations 17
Research gap and contribution 17
2.4. Review Summary 18
2.5 Problem Definition 18
Scope and implementation boundary 18
2.6 Goals/Objectives 18
2.6 Goals and measurable scope 18
2.7 Why Use These Technologies Over Others 19
Technology comparison criteria 19
STOMP over SockJS 18
MongoDB document storage 20
BCrypt password hashing 20
Frontend libraries and styling 20
Build tools and testing 20
3.1 Evaluation and Selection of Specifications/Features 21
Functional specifications 21

Non-functional specifications 21
Technology selection 21
Core feature selection 22
Data management and security 22
Final evaluation 22
3.2 Design Constraints 23
Technology and environment 23
Database and persistence 23
Real-time communication 9
Security 17
Performance and usability 23
Development and maintainability 24
Overall design considerations 24
3.3 Design Flow 24
3.3 Design Flow: frontend development 24
3.3 Design Flow: frontend modules 24
3.3 Design Flow: backend modules 25
3.3 Design Flow: database design 25
3.3 Design Flow: message transmission 25
3.3 Design Flow: room workflow 25
3.3 Design Flow: participant tracking 26
3.3 Design Flow: transient typing events 26
3.3 Design Flow: history and duplicate handling 26
3.3 Design Flow: data and service boundaries 26
3.3 Design Flow: interface 27
3.3 Design Flow: setup workflow 27
3.4 Creation of the Chatroom Application 27
Account and room setup 27
Message handling 28
Presence and typing 28
Interface behavior 28
Important algorithms 28
Message deduplication 29
Error and connection handling 29
Implementation boundary 29
3.5 Logo Designing and Naming 29
Visual identity in the interface 29
3.6 Implementation Plan/Methodology 29
Setup and execution 30
Validation approach 30
Limitations and challenges 30
4.1 Implementation of Solution 32
Report preparation and project evidence 32
Testing and data validation 32
4.1 Implementation results 32
Message delivery and persistence 33
Presence and real-time behavior 33
4.2 Validation 33
Functional validation 33
Frontend and backend validation 34
Database validation 34
Real-time and security validation 34
Performance validation 34

5.1. Conclusion 35
Project contribution and implementation limits 35
5.2. Future Work 36
Scalable message storage 37
Message delivery and moderation 37
Automated testing and operations 37
Accessibility and rich interactions 37
Privacy and data retention 37
Deployment and next steps 38
3.7 Source Code Structure and Module Responsibilities 39
3.8 REST API Design and Request Handling 40
3.9 Real-Time Messaging and Event Processing 42
3.10 Data Model and Persistence 44
3.11 User Interaction and Client State 46
4.3 Verification Strategy and Test Design 47
3.12 Local Configuration and Reproducible Execution 49
5.3 Security, Reliability and Scalability Review 51
5.4 Maintainability and Extension Design 53
3.13 Requirements Traceability and Technical Terminology 54
3.14 Worked REST API Exchanges and Contract Compatibility 55
4.4 End-to-End Communication Scenarios 57
4.5 Detailed Test Case Design 60
3.15 Source-to-Behavior Maintenance 62
5.5 Usability, Accessibility and Responsible Use 64
4.6 Acceptance Criteria and Requirement Verification 66
3.16 Data Fields, Payloads and Validation 67
4.7 Error Handling and Recovery Analysis 69
4.8 Performance Evaluation Plan 71
REFERENCES 73
PLAGIARISM REPORT 75

Detailed Contents

Project context 9
1.3.1 Requirements and planning 11
1.3.2 Interface and frontend development 11
1.3.3 Backend and API development 12
1.3.4 Data design and persistence 12
1.3.5 Authentication and integration 12
1.3.6 Real-time events 12
1.3.7 Integration and refinement 12
1.3.8 Verification and documentation 12
1.5. Organization of the Report 13
2.2.1 Centralized room communication 15
2.2.2 Stored message history 16
2.2.3 Live room events 16
2.2.4 Expected benefits 16
3.1.1 Functional specifications 21
3.1.2 Non-functional specifications 21
3.1.3 Evaluation and selection of technologies 21
3.1.4 Selection of core features 22
3.1.5 Data management and security considerations 22
3.1.6 Final evaluation 22
3.2.1 Technology and environment constraints 23
3.2.2 Database and persistence constraints 23

3.2.3 Real-time communication constraints 6
3.2.4 Security constraints 6
3.2.5 Performance and usability constraints 6
3.2.6 Development time and maintainability 6
3.2.7 Overall design considerations 6
3.4.1 Account and room setup 6
3.4.2 Message handling 6
3.4.3 Presence and typing 6
3.4.4 Interface behavior 6
3.4.5 Important algorithms 6
3.4.6 Message deduplication 6
3.4.7 Error and connection handling 6
3.4.8 Implementation boundary 6
3.6.1 Setup and execution 6
3.6.2 Validation approach 6
3.6.3 Limitations and challenges 6
4.1.1 Report preparation 6
4.1.2 Project management and communication 6
4.1.3 Testing, characterization, interpretation and data validation 6
4.2.1 Functional validation 6
4.2.2 Frontend and backend validation 6
4.2.3 Database validation 6
4.2.4 Real-time and security validation 6
4.2.5 Performance validation 6
4.2.6 Validation summary 6
5.2.1. Secure authentication and room authorization 6
5.2.2. Scalable message persistence 6
5.2.3. Distributed messaging and presence 6
5.2.4. Stronger message identity and delivery 6
5.2.5. Moderation and room controls 6
5.2.6. Expanded automated tests 6
5.2.7. Observability and operations 6
5.2.8. User experience and accessibility 6
5.2.9. Rich communication features 6
5.2.10. Privacy and retention 6
5.2.11. Deployment and scaling 6
3.7.1 Frontend entry points and routing 6
3.7.2 Landing page and room access 6
3.7.3 Chat screen and reusable components 6
3.7.4 Backend packages and responsibilities 6
3.8.1 Request conventions and API utility 6
3.8.2 Registration and login behavior 6
3.8.3 Room creation, lookup and participant endpoints 6
3.8.4 History pagination and error interpretation 6
3.9.1 Endpoint, transport and broker configuration 6
3.9.2 Message send, validation, persistence and broadcast 6
3.9.3 Join, participant update and disconnect lifecycle 6
3.9.4 Typing events, deduplication and ordering 6
3.10.1 User record and account lifecycle 6
3.10.2 Room and embedded message structure 6
3.10.3 Session state and ephemeral data 6
3.10.4 Data quality, validation and lifecycle 6
3.11.1 Account and guest identity flow 6

3.11.2 Room creation and joining sequence 7
3.11.3 Conversation loading and rendering 7
3.11.4 Composer, status and responsive behavior 7
4.3.1 Evidence boundary and testing levels 7
4.3.2 Functional and boundary scenarios 7
4.3.3 Real-time, lifecycle and concurrency scenarios 7
4.3.4 Frontend, usability and regression scenarios 7
3.12.1 Toolchain and local services 7
3.12.2 Start-up sequence and readiness checks 7
3.12.3 Configuration consistency and common failures 7
3.12.4 Build and handover procedure 7
5.3.1 Identity, trust boundaries and access control 7
5.3.2 Reliability and failure handling 7
5.3.3 Capacity, persistence growth and multi-instance operation 7
5.3.4 Operational governance and privacy 7
5.4.1 Responsibility boundaries and refactoring opportunities 7
5.4.2 Stable identifiers and event contracts 7
5.4.3 Feature extension sequence 7
3.13.1 Traceability from user need to implementation 7
3.13.2 Glossary of communication and data terms 7
3.13.3 Glossary of implementation and verification terms 7
3.14.1 Registration and login request examples 7
3.14.2 Room creation and lookup exchanges 7
3.14.3 Message history and participant exchanges 7
3.14.4 Error response handling and client mapping 7
3.14.5 API compatibility and contract maintenance 7
4.4.1 First connection to a room 7
4.4.2 Successful message exchange 7
4.4.3 Validation rejection and recovery 7
4.4.4 Leave and unexpected disconnect 7
4.4.5 History/live overlap and reconnect considerations 7
4.5.1 Account and identity cases 7
4.5.2 Room and history cases 7
4.5.3 Messaging and event cases 7
4.5.4 Participant lifecycle and concurrency cases 7
4.5.5 Browser workflow and evidence record 7
3.15.1 Changing room validation safely 7
3.15.2 Changing the message model 7
3.15.3 Changing the participant event contract 7
3.15.4 Adding an API route or user-interface feature 7
3.15.5 Review and release checklist 7
5.5.1 Labels, keyboard use and focus 7
5.5.2 Dynamic updates and status communication 7
5.5.3 Readability, responsive layout and motion 7
5.5.4 Usability study protocol 7
5.5.5 Content moderation and respectful communication 7
4.6.1 Account and identity acceptance criteria 7
4.6.2 Room and conversation acceptance criteria 7
4.6.3 Presence and typing acceptance criteria 7
4.6.4 Evidence classification and sign-off 7
3.16.1 Account request and response fields 7
3.16.2 Room, message and history fields 7
3.16.3 Transient event fields and meanings 7

3.16.4 Validation and normalization map 8
3.16.5 Serialization and compatibility notes 8
4.7.1 Browser-to-server connection failures 8
4.7.2 Validation and domain failures 8
4.7.3 Persistence and partial failures 8
4.7.4 Client recovery and state consistency 8
4.7.5 Diagnostic logging and user messages 8
4.8.1 Define a reproducible workload 8
4.8.2 Select metrics and instrumentation 8
4.8.3 Establish thresholds and interpret results 8
4.8.4 Concurrency and stability evaluation 8
4.8.5 Report performance evidence responsibly 8

LIST OF FIGURES

Fig 1 High-level application architecture 8
Fig 2 React page and component structure 8
Fig 3 REST request and response flow 8
Fig 4 MongoDB document relationship 8
Fig 5 Message broadcast and persistence sequence 8
Fig 6 Room creation and joining workflow 8
Fig 7 Participant session tracking 8
Fig 8 Typing indicator event flow 8
Fig 9 STOMP over SockJS connection flow 8
Fig 10 Chat screen interaction layout 8
Fig 11 Application setup workflow 8
Fig 12 User registration and login workflow 8
Fig 13 Message validation sequence 8
Fig 14 Testing and validation overview 8

CHAPTER 1
INTRODUCTION

1.1. Need Identification

Real-time communication is a routine requirement for study groups, project teams and online communities. In a basic chat arrangement, messages may be exchanged through delayed refreshes or separate channels, which makes conversation difficult to follow. A shared room with live delivery gives several users a common place to communicate and observe participation.

The project addresses this need through PulseChat, a browser-based multi-user chatroom application. The inspected source implements account signup and login, room creation and joining, real-time text messages, stored message history, online participant tracking, typing events and connection status. These capabilities appear together in one interface and are supported by a server that mediates communication.

Project context

A chatroom differs from a one-to-one exchange because each message is associated with a room and made available to all clients subscribed to that room. The application therefore has to coordinate room membership, concurrent sessions, data storage, user interface state and network connectivity. A robust design must also keep stored history separate from transient events such as typing status.

The users of the application include a person entering a room as a guest, a person using an account, and other browser sessions already subscribed to the same room. Their shared need is not merely to submit text, but to observe a conversation as it develops and to understand which room is active. The interface therefore combines room identity, message history, current connection state and participant feedback. Each of these concerns is represented by a different part of the implementation, so their coordination is an important part of the project problem.

A room-based conversation also creates expectations about continuity. A user who returns to an existing room expects to see recent messages, while transient activity such as typing should not be mistaken for durable conversation. The code addresses these expectations through MongoDB-backed Room and Message data for history and in-memory session state for presence. This distinction establishes a useful functional boundary: persisted records survive navigation when the database is available, whereas presence and typing describe current activity and are reconstructed or allowed to expire.

Real-time delivery and data persistence

The implementation uses HTTP endpoints for account and room operations and a STOMP messaging channel for live events. This division suits the different interaction patterns: REST requests return a finite response, while a WebSocket connection remains available for events that may arrive at any time. SockJS is configured as the transport layer for the STOMP endpoint.

A persistent store is needed so that the latest messages remain available after a user leaves and returns to a room. MongoDB is configured at localhost:27017 with the chatApp database. Room documents contain a room identifier and an embedded message list; user records are stored separately. Online participants, by contrast, are tracked in memory because they represent active server sessions rather than durable records.

The need is therefore not limited to sending text. Users also need room discovery by identifier, readable history, clear connection feedback and awareness of other participants. The project brings these interactions into one application and provides a concrete base for further improvements in security, scalability and usability.

1.1 Need Identification: User needs

Need	Implemented response
Synchronous conversation	STOMP message sent to a room destination and broadcast to subscribers
Return to earlier discussion	Room messages are stored in MongoDB and retrieved through a paginated REST endpoint

Shared group context	Room identifiers, participant list and join/leave events
Presence feedback	Participant sessions tracked in concurrent in-memory maps
Conversation cues	Typing events, connection badge, message bubbles and toast notifications
Account access	Signup/login endpoints with BCrypt password hashing

The feature set aims to make basic multi-user communication understandable without implying that rooms are access-controlled private spaces. The interface describes room creation and joining, but the inspected backend does not associate room membership with an authorization rule. Future security work must address that distinction.

1.1 Need Identification: Scope

The current scope is a single web application with a React client, a Spring Boot server and MongoDB persistence. The client supports the landing and chat routes, authentication forms, a room workflow, message history and a responsive chat layout. The server exposes authentication and room REST controllers and a STOMP message controller.

The implemented data types include User, Room and Message. A user has a display name, email, password hash and creation time. A room stores its identifier and embedded messages; each message records a sender, content and local date-time. Participant and typing events are runtime communication data and are not persisted as message history.

The project does not provide evidence of deployment to a public host, end-to-end encryption, a durable session/token mechanism, a moderation console or automated load testing. These are outside the verified scope and should not be represented as completed features.

The communication problem can be expressed through a typical interaction. A user opens a room, receives the most recent stored messages, and then remains subscribed to new events. Another user submits a message. The service must validate the content, associate it with the room selected by the destination, save it, and make it available to active subscribers. If the room identifier is missing or does not resolve, the service must not silently create a room as a side effect of sending a message.

The interface and server also have to handle partial failure. The HTTP request used to retrieve history can fail independently of the WebSocket connection, and the participant endpoint can fail without necessarily preventing history from loading. A live connection can be lost while a user is composing text. These cases require distinct states rather than one generic success/failure flag. The current client uses separate loading, history error and connection status state, which provides a foundation for communicating those differences.

1.2. Identification of Problem

Users in a group discussion need to see messages as they are sent, while returning participants need access to recent conversation context. A design based only on periodic page refresh does not provide immediate delivery and can create duplicated or stale interface state. A solution must deliver each message to the intended room participants and maintain an ordered history.

The system also faces concurrent activity: multiple browser sessions may join, send messages and disconnect independently. A participant count derived from a single global username list would be inaccurate when users share names or have more than one session. Correct cleanup on disconnect is necessary to avoid stale presence information.

In addition, communication can fail or be delayed. Users need visible connection state and the application must avoid accepting blank or oversized message content. Room identifiers and account input also require validation to reduce malformed requests.

Problem statement and security considerations

The identified problem is to build a web chat system that combines room-based real-time delivery, persistent message retrieval and session-aware presence while presenting a usable

interface. The server must distinguish persistent messages from ephemeral typing events, validate messages, store accepted content and broadcast it to room subscribers.
Account operations add another concern: email addresses should be normalized and duplicate signup should be rejected. Passwords must not be stored in plain text. The backend uses BCrypt hashing and checks a submitted password against the stored hash on login. However, the inspected login response does not issue a signed session token, so authentication should not be confused with authorization enforcement on every room operation.
The project's design addresses the core communication flow. It leaves production-grade access control, secure transport configuration and horizontal scaling as areas requiring further engineering and validation.

1.2 Problem statement and expected outcome

Problem statement: How can multiple users communicate in named rooms through a browser application, receive live messages and presence updates, and retrieve recent messages after reconnecting, while keeping the user interface responsive and the server logic organized?
Expected outcome: a working client-server application in which a user can register or log in, create or join an existing room, load the most recent messages, connect through STOMP over SockJS, send and receive room messages, observe typing and participant events, and leave the room. The backend persists accepted messages in MongoDB and exposes room history through REST.
The outcome is derived from implementation files, including ChatController, RoomController, AuthController, RoomParticipantService, WebsocketConfig, the React page components and the STOMP client hook. It is not a claim that every operational scenario has passed formal testing.
The work can be traced to concrete artifacts in the repository. The React route setup and pages provide the user journey; API modules provide account and room requests; the STOMP hook provides subscriptions and sends; Java controllers handle HTTP and broker messages; repositories and MongoDB entities provide durable storage; and RoomParticipantService provides process-local session state. This mapping is a description of the delivered structure and does not assign dates or claim a project schedule that is not present in the source history.

1.3. Identification of Tasks

The project work can be divided into requirements analysis, backend setup, data modelling, REST API implementation, WebSocket configuration, frontend workflow development, integration and documentation. Each task corresponds to a visible project component rather than a presumed development chronology.
The backend begins in ChatAppBackendApplication and is built with the Maven wrapper. Spring Data MongoDB supplies repositories; Spring Web handles HTTP requests; Spring WebSocket provides the message broker integration. Lombok generates routine accessors and constructors for entity and data classes.

1.3.1 Requirements and planning

Requirements include account registration and login, room creation and joining, message history, live message delivery, participant visibility, typing feedback and a clear path to leave a room. Technical requirements include validation, cross-origin configuration for the local frontend, MongoDB connection settings and support for concurrent WebSocket sessions.

Frontend and backend development

1.3.2 Interface and frontend development

The frontend is built with React 19 and Vite. React Router maps the root route to LandingPage and /chat/:roomId to ChatPage. Reusable components provide authentication forms, avatars, message bubbles, typing indicators, connection badges and toast notifications. CSS Modules scope styling to pages and components, while global CSS holds shared rules.
The landing workflow validates a display name for unauthenticated guests and a room identifier. Users can create a room or join an existing one. The chat page loads messages and participants, establishes the STOMP client, renders room events and provides a message composer. Enter sends a message and Shift+Enter inserts a new line.

1.3.3 Backend and API development

AuthController handles signup and login. RoomController validates room identifiers, creates rooms, returns room information, retrieves a page of messages and provides a participant list. ChatController handles message, typing, join and leave destinations. WebsocketConfig declares /chat as the SockJS endpoint, /app as the application prefix and /topic and /queue as simple broker prefixes.

Database and account management

1.3.4 Data design and persistence

The Room MongoDB document has a roomId and a list of embedded Message objects. Message records sender, content and time. User documents contain name, unique indexed email, password hash and createdAt. UserRepo and RoomRepo provide repository access. Message history is read from the embedded room list and returned in chronological order within each page.

The application stores room and message data durably in MongoDB. Participant session maps are held in a RoomParticipantService using ConcurrentHashMap-backed collections and are cleared when sessions disconnect. The configured default Mongo URI points to a local instance; deployment configuration is not present in the inspected files.

1.3.5 Authentication and integration

Signup normalizes email to lowercase, validates the display name, email format and a minimum password length, rejects duplicate email and stores a BCrypt hash. Login retrieves by normalized email and checks the password hash. The returned AuthResponse contains identity details, but no JWT or durable login token is generated by the inspected controller.

Real-time events and integration

1.3.6 Real-time events

The STOMP client subscribes to room messages, typing events and participant updates. It sends messages to /app/sendMessage/{roomId}, typing state to /app/typing/{roomId}, and join/leave notices to the corresponding application destinations. The server uses the room path as the authoritative room identifier for message handling.

Participant registration maps a WebSocket session to a room and username. A concurrent set maintains sessions per room. Join and leave events publish the current participant count and distinct user names. A disconnect listener removes sessions that close without an explicit leave.

1.3.7 Integration and refinement

Integration connects the React API modules to REST endpoints and the STOMP hook to the WebSocket endpoint. Message history is merged with live events using a key composed of sender, timestamp and a content prefix to reduce duplicates. The interface tracks loading, error and connection state and offers retry or navigation behavior for failed room loading.

Verification and documentation

1.3.8 Verification and documentation

The repository contains a Spring context-load test with repository and messaging dependencies mocked. It is a narrow startup test, not a comprehensive suite for authentication, room validation, message ordering, concurrent sessions or frontend behavior. No recorded performance measurements or manual test report was included in the project folder.

Documentation tasks include recording prerequisites, configuration, local startup steps, API destinations, data flow, implementation boundaries and future work. A reproducible report should distinguish verified source behavior from recommendations that remain unimplemented.

1.4. Timeline

The source folder does not provide dated commits, a project schedule or a formal work log. The following timeline is therefore an implementation sequence suitable for describing the work, not a claim about calendar dates or measured duration.

Schedule evidence and report structure

Phase	Work products
1. Requirement definition	User needs, functional scope and system

	boundaries
2. Technology setup	Java 21, Spring Boot, Maven wrapper, React/Vite dependencies and MongoDB configuration
3. Backend foundation	Entities, repositories, account endpoints and room endpoints
4. Real-time layer	SockJS/STOMP endpoint, broker destinations, chat controller and presence tracking
5. Frontend workflow	Landing page, authentication, room flow, chat view and reusable interface components
6. Integration	REST history combined with WebSocket updates; connection and error states
7. Review	Source inspection, context-load test definition and report preparation

A project diary or version-control history would be required to confirm when each phase occurred and how long it took.

1.4 Timeline and 1.5 Report organization

1.5. Organization of the Report

Chapter 1 introduces the need, problem, tasks, timeline and organization. Chapter 2 reviews the communication background, proposed solution, related technology concepts, problem definition, objectives and selection rationale. Chapter 3 describes requirements, design constraints, architecture, data and event flows, implementation plan and setup. Chapter 4 summarizes implemented behavior and validation evidence. Chapter 5 presents conclusions and proposed future work. References identify the official technical documentation consulted for terminology and framework descriptions.

The chapter labels and sequence in this report follow the body structure of the supplied report. The contents page in the PDF described a different chapter outline, so its entries were reconciled to the headings actually present in the report body.

CHAPTER 2
BACKGROUND STUDY

2.1. Timeline of the Reported Problem

Networked messaging evolved from asynchronous message exchange toward persistent connections and event-driven delivery. Modern browser applications can maintain a bidirectional channel through WebSocket, while a messaging protocol such as STOMP provides named destinations and publish-subscribe semantics above the transport.

For a room chat, the central challenge is coordinating several client sessions that share a topic. A server-side broker routes published messages to subscribed clients. Durable history remains a separate concern: a broker delivers live events but does not by itself provide an archive. PulseChat therefore combines a messaging broker with MongoDB-backed room data.

The need for ongoing communication arises because an HTTP request/response exchange is naturally finite. A client can ask for a page of messages and receive a response, but repeated polling would require the browser to ask again even when nothing changed. A persistent messaging connection gives the server a path to publish an event when a new message or room update occurs. This is useful for a chatroom because several clients can be waiting for the same room topic at the same time.

The project separates this live channel from its history interface. REST is used for account operations, room creation and lookup, message history and participant reads. STOMP over SockJS is used for message, typing, join and leave events. The split makes the responsibilities visible: a finite query returns a snapshot, while a subscription stays available for future events. Neither channel replaces the other; history supports reload and catch-up, while the subscription supports interaction during an active session.

Persistent data and session state

A typical chat service separates durable entities from transient state. Accounts, rooms and messages may survive a restart; typing status and current online presence usually do not. The project reflects this distinction by storing user and room documents while tracking online sessions in memory.

Concurrent participation also requires safe shared state. RoomParticipantService uses concurrent maps and sets so separate WebSocket event handlers can update session membership without relying on a single mutable list. This supports concurrent access within one server process, although it does not synchronize state across multiple server instances.

Browser and server architecture

Modern applications commonly divide responsibilities between a browser client, an HTTP API, a real-time channel and a persistence layer. The client presents user interaction; REST handles finite operations such as creating a room; WebSocket handles ongoing events; MongoDB stores durable records.

This separation helps explain the project architecture and its limits. The local configuration demonstrates development-time integration. It does not establish deployment readiness, reliability under load or operational security in a public environment.

Frontend and backend frameworks

Browser-side frameworks make it practical to express the interface as reusable components and state transitions. React manages updates when messages arrive, when a user types, and when a connection changes. Vite supplies the frontend development and build workflow. CSS Modules isolate presentation styles.

On the server, Spring Boot organizes HTTP controllers, repositories, services and WebSocket configuration. Spring Data MongoDB maps Java objects to documents. The framework structure makes components easier to inspect, but careful validation and test coverage remain necessary.

History retrieval and duplicate handling

A chat client must reconcile the initial history response with later live messages. The project loads recent messages through REST and subscribes to the room topic for new events. The

frontend derives a message key from sender, time and content prefix to avoid duplicate insertion when overlapping data appears.
This approach is pragmatic for the current prototype. A stable server-generated message identifier would provide more reliable deduplication, especially when timestamps collide or message text shares the same prefix.

Presence and concurrent sessions

Presence is an approximation of active transport sessions. The backend associates a session id with a room and username, publishes JOIN and LEAVE events, and removes membership on disconnect. A displayed participant count therefore reflects active tracked sessions within the current server process.
It should not be interpreted as a permanent user directory or a guarantee that a browser remains actively attended. Reconnection handling, session expiry and multi-instance coordination require additional design.
The architecture can be understood as two complementary interaction cycles. The first is a request cycle: the browser asks the REST API for a room, account operation or history page and receives a finite response with an HTTP status. The second is an event cycle: the browser establishes a SockJS connection, negotiates STOMP, subscribes to room destinations and receives events until the connection ends. Keeping these cycles distinct makes recovery understandable. A failed history request can be retried without recreating the room, while a lost broker connection can be reported separately from an HTTP error. The client code reflects this division through its API helpers and the chat hook rather than using one transport for every operation.
The room identifier connects the cycles. It is used by the route, by the room REST endpoints, by the STOMP application destination and by the topic subscription. The message handler takes the destination room identifier as authoritative when locating the Room document. This convention helps prevent a payload from naming a different room than the one addressed by the send destination. It does not establish authorization: the current handler does not verify that the sender is a member or authenticated principal. A production design would derive identity and room access from server-verified credentials, but that is a future requirement rather than a delivered capability.

2.2. Proposed Solution

The proposed solution is the implemented PulseChat web application: a React client communicates with a Spring Boot backend over REST and STOMP over SockJS. MongoDB stores users, rooms and room message history. The design combines room-based delivery with a single chat interface and separate controls for account and room workflows.
A user can register or log in, choose to create or join a room, and navigate to its chat route. The client requests recent history and participant information, connects to the room topics and sends messages through an application destination. The server validates, stores and broadcasts each accepted message.
The proposed system is organized as a React client, a Spring Boot application and a MongoDB data store. The browser handles form interaction, route selection and local presentation state. The backend validates requests, performs room and account lookups, writes accepted messages and routes room events. MongoDB stores User and Room documents, with messages embedded inside their Room. The simple broker routes live events within the running application process.

Room and history capabilities

2.2.1 Centralized room communication

Each room has a textual identifier. Room creation accepts identifiers from 3 to 50 characters consisting of letters, digits, underscores and hyphens, and returns conflict when the identifier already exists. Joining checks that the room exists. These rules help users share a room id, but do not make a room private or require an invitation.

2.2.2 Stored message history

Messages are appended to the room document and saved through RoomRepo. A REST history request defaults to page zero and size 50; size is clamped to 1-200. Page zero selects the newest records and returns them oldest-first. Older pages are selected by increasing the page number.

Live events and expected benefits

2.2.3 Live room events

STOMP subscriptions use /topic/room/{roomId} for chat messages, /typing for typing state and /participants for participant updates. The client sends to /app/sendMessage/{roomId}, /app/typing/{roomId}, /app/join/{roomId} and /app/leave/{roomId}. This convention separates client requests from broker topics.

Typing events are broadcast without database persistence. Participant events are generated by a server-side service and are likewise ephemeral. Chat messages are durable and appear in history after retrieval.

2.2.4 Expected benefits

The architecture provides immediate room updates, a retrievable recent history, clearer presence feedback and distinct responsibilities across client, controller, service and repository layers. These are design benefits; user satisfaction, throughput and latency have not been measured in the supplied project.

User feedback and operational boundaries

The application handles failed operations with validation messages, toast notifications, room-not-found behavior and a connection badge. The chat interface disables sending while disconnected, supports Enter-to-send and Shift+Enter for multiline content, and includes a scroll-to-bottom affordance when a reader moves up through history.

The project also includes a signup and login form. Email addresses are trimmed and lowercased, duplicate emails are rejected, and passwords are hashed with BCrypt before persistence. However, there is no token issuance or room-level access check in the inspected implementation; the user interface login state alone should not be considered strong authorization.

Local configuration and deployment scope

The solution is intended for local development and demonstration. The application properties point to a local MongoDB instance, the backend uses port 8081 and the Vite development server conventionally uses port 5173. CORS and WebSocket origins include local development addresses.

For deployment, environment-specific configuration, HTTPS/WSS, secure session handling, authorization, rate limits, monitoring, backup and a production broker/database strategy would need to be designed and verified. These are proposed improvements rather than current capabilities.

2.3. Bibliometric Analysis

A formal bibliometric study requires a documented search strategy, databases, date range, inclusion criteria and analyzed publication set. None was supplied with the project source. This section therefore provides a qualitative technology and literature context, not publication counts or quantitative citation statistics.

Research and engineering writing on real-time web applications commonly discusses persistent connections, publish-subscribe messaging, concurrent state, durable storage and user experience. Those themes map directly to the design choices in this project.

The technology review is qualitative and based on the project's source and official documentation referenced in the report. It compares technologies by role in this implementation rather than asserting a universal ranking. REST is suitable for finite requests such as sign-in and history retrieval. STOMP provides named destinations and subscriptions for room events. SockJS is configured as the browser/server transport library around the /chat endpoint. MongoDB is used for document persistence, while React components update the browser interface as asynchronous data arrives.

For a chat application, message delivery and message retention are different problems. A broker can distribute an event to current subscribers, while an application database can preserve that event for a later history request. The project uses both mechanisms: Spring's simple broker publishes room topics, and RoomRepo saves a changed Room document. This combination is appropriate for demonstrating a full-stack prototype, but it should not be confused with a broker that durably retains events or coordinates multiple backend nodes.

WebSocket and STOMP messaging

WebSocket is a standardized protocol for bidirectional communication between a client and server over a single connection. STOMP supplies a frame and destination model that allows clients to subscribe and publish messages. SockJS provides a client/server library intended to support WebSocket communication with fallback transports where needed.

In this project, Spring’s simple message broker routes topic messages within the application process. The configuration uses /topic and /queue broker prefixes, /app for application destinations and /user for user destinations. No external message broker is configured in the inspected files.

Publish-subscribe design

Publish-subscribe messaging is useful when one event should reach several interested clients. A room topic represents that shared audience. The server maps the room-specific application destination to its room topic, persists the message, and returns the message for broadcast.

This pattern is distinct from a request-response API. An HTTP request completes with a response; a topic subscription remains active for future events. Keeping the two flows separate clarifies message history versus live delivery.

Document storage model

Document databases organize records as flexible documents. The project stores each room and its messages in MongoDB, embedding messages within the room document. This simplifies loading and saving a room’s history for a small application, while large unbounded message arrays may become a scaling concern.

A future design could store each message as a separate document with a room index and timestamp, enabling more scalable pagination and retention policies. The present implementation uses the embedded list and should be described accordingly.

Component-based interface design

Component-based interface design encourages reuse and predictable rendering. The application has dedicated components for messages, avatars, connection state, typing status and authentication. The chat page coordinates data loading and real-time subscriptions, while the API modules isolate HTTP calls.

These choices reduce duplication and help locate responsibilities. The inspected frontend uses JavaScript JSX; TypeScript appears in the TypeScript configuration and package dependency, but the application source files shown are JSX rather than TypeScript.

Security considerations

Security guidance for web applications emphasizes password hashing, input validation, transport protection and authorization checks. The project implements BCrypt password hashing and validation for selected inputs. It does not demonstrate end-to-end encryption, HTTPS deployment, role-based access or authenticated WebSocket authorization.

The project’s account API is a useful foundation, but production use requires a security review. In particular, login success is not evidence that later room and message operations are protected against impersonation.

Research gap and contribution

For the chatroom context, the practical gap addressed by this project is integration: live delivery, stored recent history and participant awareness are assembled in one browser workflow. The contribution is an implemented prototype that demonstrates these concepts with a Java/Spring backend and a React frontend.

The project does not claim novel messaging protocols or publish-subscribe algorithms. Its academic value is in requirements analysis, system integration, implementation and reflection on limitations.

2.4. Review Summary

The background review establishes the distinction among request-response communication, real-time event delivery and persistent storage. PulseChat uses REST for account and room actions, STOMP over SockJS for ongoing room events, and MongoDB for durable account, room and message records.

The system also tracks active sessions in concurrent in-memory structures and broadcasts participant changes. This makes presence responsive within one process while making it transient across a server restart. The design is suitable for a learning prototype and exposes clear opportunities for stronger security, scalable persistence and automated evaluation.

2.5 Problem Definition

The problem is to implement a multi-user web chat application where multiple independent clients can join named rooms, exchange text messages in real time, review recent history and receive participant and typing feedback. The application must validate user input, persist messages, indicate connection status and clean up participant state on disconnect.

The problem includes user workflow as well as server correctness. A room identifier must be validated and checked for conflicts; a message must have a sender and nonblank content within the configured maximum length; a room must exist before a message can be stored and broadcast. Concurrent clients must subscribe to the correct room destination.

Scope and implementation boundary

The scope is bounded by the project code: browser-based client, Spring Boot backend, local MongoDB configuration, REST endpoints, SockJS/STOMP messaging, and a single-process in-memory participant service. Authentication endpoints validate credentials and hash stored passwords, but the implementation does not create a signed token or enforce authenticated identity on chat messages.

The system is evaluated from source evidence and the repository's test definition. No claims are made about measured latency, capacity, security certification or live deployment. This boundary keeps the report accurate and identifies meaningful future work.

2.6 Goals/Objectives

- Provide a browser interface for users to register or log in.
- Allow users to create rooms and join existing rooms by identifier.
- Deliver text messages to clients subscribed to the same room.
- Persist room messages in MongoDB and expose paginated history.
- Track connected WebSocket sessions and publish participant updates.
- Broadcast typing state as a transient room event.
- Expose connection, loading and error state in the user interface.
- Organize code into frontend components and backend controller/service/repository layers.

2.6 Goals and measurable scope

Objective area	Evidence in source
Account handling	AuthController signup/login; BCryptPasswordEncoder; unique email index
Room lifecycle	RoomController create, lookup and participant endpoints
Message delivery	ChatController @MessageMapping and @SendTo
Persistence	Spring Data MongoDB repositories and Room document
Presence	RoomParticipantService and disconnect event listener
Interface	LandingPage, ChatPage, connection and

	message components
Verification	One Spring context-load test; no broader recorded test results

These objectives describe implemented or inspectable behaviors. Items such as invitation privacy, authenticated room membership, message editing and mobile-native support are not included in this list because they are not implemented.

The choice of MongoDB is consistent with the current document model. A Room stores its logical identifier and an embedded list of Message objects, and each message has sender, content and time fields. This representation makes a room and its conversation history available through one repository lookup. It also introduces a growth constraint: the complete embedded list can become large, and the history controller slices data after obtaining the room. A relational schema or a separate message collection could offer different indexing and pagination options, but changing storage would require a migration and query redesign. The present choice should therefore be evaluated against the size and retention limits of the intended small-room use case.

The selected communications stack also reflects the framework already used by the project. Spring's message broker configuration maps application sends and subscriptions to explicit prefixes, while STOMP provides named destinations and message frames over the SockJS endpoint. A raw WebSocket protocol could carry events but would require the application to define more of its own routing conventions. Polling over REST would simplify infrastructure but introduce repeated requests and slower update discovery. The selected arrangement gives the project a clear event model, while the simple broker remains process-local and does not provide a distributed queue or cross-instance coordination.

2.7 Why Use These Technologies Over Others

React was selected for a component-driven interface with state updates when network events arrive. Vite provides a development server and frontend build pipeline. The source currently uses React JSX and CSS Modules, which offer scoped component styling without a utility CSS framework.

Spring Boot brings HTTP and WebSocket support in a single Java application. Spring Data MongoDB maps entities and repositories to the document store. Maven Wrapper supplies a project-specific build entry point without requiring a separate Maven installation.

Technology comparison criteria

Technology	Role in this project	Reason in context
Java 21	Backend language	Typed server-side code and Spring ecosystem
Spring Boot 3.4.2	Application framework	Organizes REST, WebSocket, configuration and dependency wiring
Spring Web	REST API	Handles account and room HTTP endpoints
Spring WebSocket	Messaging	STOMP endpoint and simple broker integration
MongoDB	Persistence	Stores user and room documents with embedded message history
React 19 / Vite	Frontend	Component-based browser interface and dev/build workflow

STOMP over SockJS

STOMP over SockJS is used for the room’s event channel. STOMP names destinations and supports subscriptions, while SockJS is configured at /chat. This is more expressive for room subscriptions than manually encoding all event routing in raw socket frames.

A simple broker is enabled in the Spring configuration. It is an in-process broker suitable for development and modest demonstrations. An external broker would be considered for larger deployments or coordinated multi-instance delivery.

MongoDB document storage

MongoDB is used rather than a relational database in the current implementation. Room records embed message lists, and User documents store account fields. The schema is straightforward for a prototype, but an unbounded embedded history may need redesign when rooms accumulate many messages.

The configuration uses a local MongoDB URI and the database name chatApp. There is no evidence of a hosted database service, migrations, or a formal backup process.

BCrypt password hashing

BCrypt is used for password hashing through Spring Security Crypto. It is a password-hashing function, not encryption that can be reversed to recover the original password. The application checks the submitted password with the stored hash during login.

Authentication is not complete access control. The source does not issue JWTs or apply a security filter to require a logged-in principal for room APIs or STOMP messages. The report therefore distinguishes password hashing from authorization.

Frontend libraries and styling

The frontend uses React Router for page navigation, @stomp/stompjs and sockjs-client for messaging, and lucide-react for icons. Vite and React are declared in package.json. CSS Modules and global CSS are used for presentation; Tailwind CSS and TypeScript are not the primary implementation approach evident in the source files.

This selection matches the current code and avoids attributing packages or frameworks that are not declared or used.

Build tools and testing

Lombok reduces routine Java boilerplate for data classes and constructor injection. The backend is built with Maven Wrapper and includes Spring Boot Test dependencies. A small context-load test is present, though the project’s quality assurance needs broader unit and integration coverage.

Technology choices are assessed here in relation to the implementation. No comparative benchmark was conducted, so the report does not assert that these technologies are universally faster or superior to alternatives.

CHAPTER 3
DESIGN AND PROCESS

3.1 Evaluation and Selection of Specifications/Features

The selected features follow the main user journey: account access, room creation or joining, chat connection, message exchange, history retrieval and room exit. Supporting status feedback helps users understand whether the real-time channel is ready. Participant and typing updates provide additional context without being stored as conversation messages.

The functional requirements are linked to observable operations. Account requirements map to signup and login endpoints. Room requirements map to create and lookup operations. Messaging requirements map to a validated STOMP send and a room-topic broadcast. History requirements map to a paginated REST read. Presence and typing requirements map to separate transient event types. Connection awareness maps to the STOMP hook status and the ConnectionBadge. This traceability makes it possible to locate the implementation for each requirement and to design a focused test.

Functional specifications

3.1.1 Functional specifications

ID	Requirement	Implemented behavior
F1	Register and log in	Signup/login REST routes with input validation and BCrypt
F2	Create room	POST room endpoint validates identifier and detects duplicates
F3	Join room	GET room lookup returns not found for unknown room
F4	Send message	STOMP handler validates, persists and broadcasts
F5	Read recent history	Paginated REST endpoint returns chronological page
F6	Show presence	Session-aware participant events and list endpoint
F7	Show typing	Transient typing event broadcast by room
F8	Show connection state	Frontend connection badge and guarded composer

Non-functional specifications

3.1.2 Non-functional specifications

Usability requires clear room workflow, readable message bubbles, visible connection state, responsive layout and nonblocking feedback. Maintainability is supported by separated API modules, reusable React components and backend controller/service/repository layers. Concurrent access is considered in participant tracking through concurrent maps and sets.

Reliability depends on MongoDB availability for account, room and message operations and on the WebSocket connection for live messaging. Scalability is limited by the in-process simple broker, in-memory presence state and embedded message arrays. Security is partial: password hashing and selected validation exist, but production authorization and secure transport are not demonstrated.

Technology selection

3.1.3 Evaluation and selection of technologies

The project uses Java 21 and Spring Boot 3.4.2 in the backend. The dependencies include Spring Web, Spring WebSocket, Spring Data MongoDB, Spring Security Crypto and Lombok. The

frontend package declares React 19, Vite, React Router, STOMP.js, SockJS client and Lucide React.
The repository's package.json and pom.xml are the authoritative version sources. The Vite config, source imports and application properties establish the development setup. The implementation is JavaScript JSX; TypeScript is present as a package dependency/configuration but no TypeScript application source was found in the inspected file inventory.

Core feature selection

3.1.4 Selection of core features

The core feature set prioritizes communication and room context: sending messages, receiving broadcasts, retrieving history, tracking participants and showing connection state. Account flows provide a basic identity record. The design keeps typing and presence ephemeral because they describe current activity rather than durable conversation content.
The frontend also supports message deduplication, automatic scroll behavior, an explicit jump-to-bottom button, toast messages and responsive sidebar behavior. These are usability details supported in ChatPage and component files.

Data management and security

Data handling follows a small set of explicit transformations. Signup normalizes the email before checking for duplicates and persisting the account, then stores a BCrypt-encoded password rather than the submitted password value. Login applies the same email normalization and verifies the submitted password against that encoded value. These are concrete protective measures in the account workflow. The implementation does not issue a signed access token, apply a session-based authentication filter to chat destinations, or authorize room membership. Consequently, the password hash should not be interpreted as proof that later WebSocket messages are linked to the registered account.
Message persistence uses the Room document as the aggregate boundary. The handler resolves the room, constructs a Message with a timestamp and appends it to the room's list before publishing the message to subscribers. This ordering means that a message delivered through this handler has first passed the room lookup and save operation. The source does not implement a transaction spanning MongoDB and broker delivery, nor a durable outbox. If the database save succeeds and broker publication fails, a later history read may reveal a message that active subscribers did not immediately receive. Conversely, a client retry after an uncertain connection result could submit a duplicate because messages do not have a stable client or server idempotency key.

3.1.5 Data management and security considerations

User email has a unique index. Signup converts email to lowercase, validates a regular-expression pattern, checks duplicates, hashes the password and returns a response that omits the password. The Room entity stores embedded Message records. Message content is trimmed and limited to 2000 characters before persistence.
The application does not demonstrate database encryption, end-to-end encryption, JWT, durable session tokens, role-based permissions, or authenticated WebSocket principals. Room ids are identifiers, not access credentials. These constraints should be addressed before handling sensitive or public traffic.

Final evaluation

3.1.6 Final evaluation

The selected design satisfies the prototype's central objective: room-scoped live chat with persistent recent history and presence feedback. It uses a modest set of technologies that map directly to the required interactions.
The design is not a production deployment blueprint. Message volume, throughput, latency, memory behavior and security posture have not been empirically measured. The test source checks Spring context wiring only.

3.2 Design Constraints

Design constraints arise from the local configuration, framework choices, single-process broker, embedded data model and absence of a deployment environment. The system also depends on a running MongoDB instance and browser access to the configured backend and frontend origins.

A browser client, REST API and WebSocket connection operate across separate network boundaries. The browser can be refreshed or closed, requests can fail independently, and the server can be restarted while persistent data remains in MongoDB. The design therefore treats connection state as explicit client state and participant state as process-local server state. It does not assume that a browser tab is continuously connected or that the in-memory service survives a backend restart.

Technology and environment

3.2.1 Technology and environment constraints

The backend requires Java 21 as configured in the Maven project. The frontend uses Node.js and npm for the Vite workflow. The Maven wrapper is included. The application properties configure the backend on port 8081 and connect MongoDB at localhost:27017/chatApp.

WebSocket and CORS configuration explicitly allow selected localhost origins. A different host or port requires configuration changes. No environment variable-based production profile was identified in the inspected files.

Database and persistence

3.2.2 Database and persistence constraints

A room document contains an embedded list of messages. This makes a room's data easy to load as one aggregate, but message arrays can grow indefinitely. The history endpoint slices the in-memory list and caps page size at 200; it does not use database-side message pagination.

MongoDB is required for account, room and message operations. If it is unavailable, durable operations cannot complete. The local URI and lack of backup/retention configuration constrain this project to development-scale use.

Real-time communication

3.2.3 Real-time communication constraints

The simple message broker and participant maps are local to one backend process. If traffic is distributed across several instances, subscribers and presence records will not automatically be shared. A broker relay or another distributed messaging approach and coordinated session state would be required.

Network disconnects can interrupt live messaging. The disconnect event listener removes tracked participants, but reconnect synchronization and duplicate-session behavior need broader testing. A live message is received only while the client has an active subscription; persistence supports later history retrieval.

Security

3.2.4 Security constraints

Passwords are BCrypt-hashed and input checks exist for account fields, room ids and message content. However, the backend does not establish authenticated identities for STOMP messages or protect room endpoints with a security policy. A sender name is included in the message request and should not be trusted as proof of identity.

Local HTTP endpoints do not demonstrate TLS. Before deployment, the application needs HTTPS/WSS, authorization, abuse controls, secret management, error handling review and security testing.

Performance and usability

3.2.5 Performance and usability constraints

The application has no recorded load or latency measurements. Embedded message history may increase document size and serialization cost. In-memory participant tracking consumes resources proportional to active sessions. The UI has auto-scroll behavior and a scroll-to-bottom control, but very long histories may require virtualization or incremental loading.

These are anticipated constraints based on the design, not observed benchmark outcomes.

Development and maintainability

3.2.6 Development time and maintainability

The repository includes a modular structure but no formal schedule, requirements traceability file or release history in the inspected folder. Maintaining alignment across REST paths, WebSocket destinations, DTOs and frontend hooks requires coordinated changes.

An API contract, stronger automated tests and consistent configuration profiles would improve maintainability. The current context test does not validate business rules or client behavior.

Overall design considerations

3.2.7 Overall design considerations

The design favors understandable separation of roles: React components manage interaction, API modules call REST endpoints, Spring controllers handle requests, a participant service coordinates session state, repositories access MongoDB and the simple broker distributes events.

Future changes should keep message persistence, authorization and event routing explicit. Growth beyond a single-process demonstration will require revisiting storage, broker and participant coordination.

3.3 Design Flow

The architecture has four principal parts: the browser interface, REST controllers, the STOMP message channel and MongoDB. The user interacts with the browser, which calls REST endpoints for account, room and history operations and uses a persistent SockJS/STOMP connection for live events.

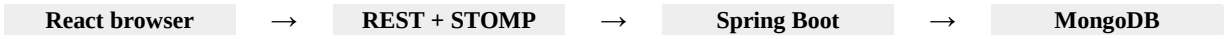


Fig 1. High-level application architecture

At runtime, a room interaction begins with a route containing roomId and a display name available from navigation state or local storage. ChatPage starts its history and participant reads and the STOMP hook establishes a client connection. After connection, the client subscribes to message, typing and participant topics and publishes a join event. These operations can complete in different orders, so the page must treat loading, connection and participant state as related but separate signals.

A message then follows a server-mediated path. The composer checks local state and calls the hook. The client publishes a JSON payload to an application destination containing the room identifier. ChatController validates text and sender, resolves the room from the destination, creates the timestamped object, saves the updated room and returns the message. Spring sends that result to the room topic. Each subscribed client parses the frame and passes it to ChatPage for duplicate checking and rendering.

3.3 Design Flow: frontend development

The React application mounts in main.jsx and renders App. BrowserRouter declares the landing route and room chat route. ToastProvider and AuthProvider wrap the routes. LandingPage manages account presentation and create/join selection; ChatPage manages history, participants, messages, typing state and connection status.

The frontend API modules encapsulate HTTP calls. The STOMP hook handles connection lifecycle, subscriptions and publish operations. MessageBubble, Avatar, TypingIndicator and ConnectionBadge render focused interface elements. CSS Modules scope page and component styles.

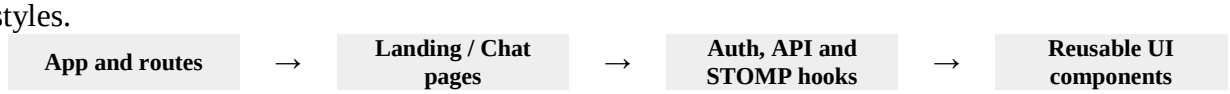


Fig 2. React page and component structure

3.3 Design Flow: frontend modules

Module	Responsibility
App.jsx	Routes and context providers
LandingPage.jsx	Account-aware landing screen and room actions

ChatPage.jsx	History load, event handling, composition and display
roomApi.js	Room REST requests
authApi.js / AuthContext.jsx	Authentication requests and client auth state
useStompClient.js	STOMP/SockJS connection and room subscriptions
useTypingIndicator.js	Typing start/stop timing
MessageBubble / ConnectionBadge	Reusable message and status rendering

3.3 Design Flow: backend modules

AuthController implements signup and login under /api/v1/auth. RoomController implements creation, room lookup, message history and participant listing under /api/v1/rooms. ChatController handles STOMP destinations for sending messages, typing, joining and leaving.

RoomParticipantService tracks session-to-room and session-to-user mappings and room-to-session sets. WebSocketEventListener cleans up a session on disconnect. WebsocketConfig registers the /chat SockJS endpoint, configures allowed local origins, sets destination prefixes and enables an in-process simple broker. Repositories persist Room and User entities.

Browser action → **REST endpoint** → **Controller validation** → **Repository / MongoDB** → **JSON response**

Fig 3. REST request and response flow

3.3 Design Flow: database design

Document	Fields / contents	Persistence role
User	id, name, email, password hash, createdAt	Account record; email uniquely indexed
Room	id, roomId, embedded messages[]	Room record and chat history
Message	sender, content, time	Embedded in the room document
Participant state	session-room/user maps and room session sets	In-memory only; cleared on disconnect

This model distinguishes stored conversation data from live presence. The participant maps do not survive a process restart. The room identifier is validated by RoomController and used by the message handler to find the room.

User document → **Room document** → **Embedded Message list** → **Participant state in memory**

Fig 4. MongoDB document relationship

3.3 Design Flow: message transmission

Composer → **STOMP SEND** → **Validate and resolve room** → **Save to MongoDB** → **Broadcast to room subscribers**

Fig 5. Message broadcast and persistence sequence

The client trims input and prevents blank sends. ChatController checks for nonblank content, enforces a 2000-character limit, requires a sender and verifies that the target room exists. It creates a timestamped Message, appends it to the room and saves the room before returning the message for broadcast.

The destination room id is taken from the STOMP path variable rather than trusting a potentially inconsistent room id in the payload. This is an input consistency measure; it does not authenticate the claimed sender.

3.3 Design Flow: room workflow

Landing page → **Create or join request** → **Validate id / find room** → **Open /chat/{roomId}**

Fig 6. Room creation and joining workflow

Room creation accepts JSON or plain text input and extracts the room identifier. It requires 3-50 characters and permits letters, numbers, hyphens and underscores. Duplicate identifiers return a conflict response. Joining retrieves an existing room or returns not found.

The client may generate a suggested identifier from a fixed adjective/noun list and a three-digit number. The generated value is a convenience, not a cryptographic random token or room access secret.

3.3 Design Flow: participant tracking

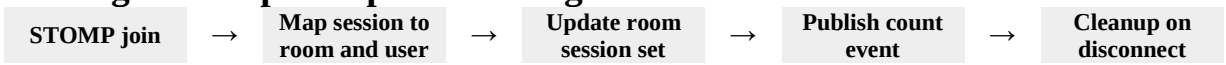


Fig 7. Participant session tracking

On join, the server registers the WebSocket session id, room id and username, then broadcasts a participant event. On explicit leave or transport disconnect, it removes the mapping and publishes an updated event. getParticipants returns distinct sorted usernames, while the event count is based on active sessions.

If a user opens multiple sessions, the count may exceed the number of distinct names. The UI currently uses participant list length or the event count depending on the display location.

3.3 Design Flow: transient typing events

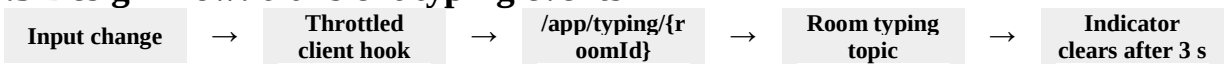


Fig 8. Typing indicator event flow

Typing events are not persisted. The frontend suppresses its own typing state, tracks active senders and clears stale indicators after three seconds. The typing hook throttles event emission to avoid sending on every keystroke. This event path is separate from chat message history.

3.3 Design Flow: history and duplicate handling

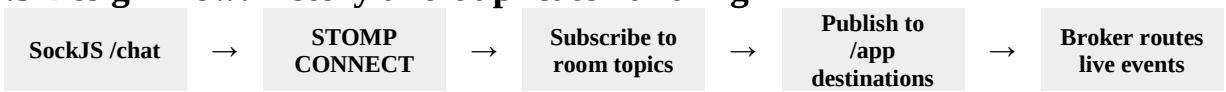


Fig 9. STOMP over SockJS connection flow

When a room loads, the client requests the first history page and the current participant list. The returned messages initialize the visible state and a set of seen keys. Incoming live messages are checked against that set and against current message state before append.

The key combines sender, time and the first 40 characters of content. A server-side unique message id would be a stronger long-term identity strategy.

The service boundaries are useful for reasoning about change. Controllers translate transport data into operations: AuthController handles account requests, RoomController handles room and history requests, and ChatController handles STOMP message and room-event destinations. Repositories isolate document lookup and save operations. RoomParticipantService holds connection-scoped presence mappings and emits participant snapshots. On the browser side, API modules isolate HTTP paths, the STOMP hook isolates connection and subscription mechanics, and React pages coordinate form state and presentation. This division allows a change to one transport adapter to be reviewed separately, provided the shared request and event contracts remain compatible.

There are still contracts that cross module boundaries and deserve explicit attention. The room path parameter must use the same normalized identifier as the stored room. A message frame must carry the fields expected by the backend DTO, and the returned frame must match the shape rendered by the client. Participant snapshots must expose a collection that the UI can sort or display consistently. These contracts are currently represented in code rather than a generated schema. A future improvement could define request and event schemas and validate them at both ends, reducing the chance that a field rename compiles successfully in one language while failing only at runtime in the other.

3.3 Design Flow: data and service boundaries

MongoDB stores user documents and room documents with embedded messages. The simple broker routes events for active subscribers. Participant tracking resides in concurrent in-memory collections; typing event payloads pass through the broker and are discarded after delivery.

The separation clarifies restart behavior: chat history remains in MongoDB while live participant and typing state resets when the backend process stops.

3.3 Design Flow: interface

The landing screen presents PulseChat identity and account access, then supports room creation or joining. In the chat screen, a sidebar displays the room identifier, current user, online participants, connection status and a leave action. The main area shows room messages, typing cues, connection state and a composer.

The UI supports a collapsible sidebar for smaller screens, toast notices for network and participant events, color-coded initials avatars, auto-scroll while near the bottom, and a jump button after scrolling upward. These descriptions are based on the inspected JSX and CSS modules.

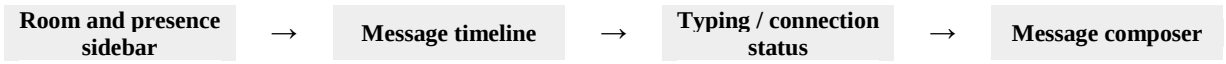


Fig 10. Chat screen interaction layout

3.3 Design Flow: setup workflow



Fig 11. Application setup workflow

The project README requires Java 21, Node.js, npm and MongoDB. The backend can be launched with the included Maven wrapper from chat-app-backend. The frontend is installed and launched with npm from chat-app-frontend. The frontend API and WebSocket URLs can be configured with VITE_API_BASE_URL and VITE_WS_URL as documented in the README.

The committed application.properties value sets server.port=8081 and the local Mongo URI. Configuration should be checked against the running service rather than relying on stale documentation.

3.4 Creation of the Chatroom Application

The application is assembled from a browser client, a Spring Boot server and MongoDB. The backend entry point enables Spring Boot component discovery. Configuration classes enable cross-origin REST requests, register the SockJS endpoint and manage WebSocket lifecycle events.

The frontend is delivered as a Vite project. App.jsx defines the route structure and wraps it in authentication and toast contexts. Page-level components coordinate user actions and shared components display local interaction state.

The frontend and backend divide validation work. The landing page checks room identifier syntax and guest name length before sending a request, giving immediate feedback. RoomController repeats the authoritative room checks, trims the identifier and handles conflict or lookup results. AuthController normalizes email and validates account fields at the server. Message validation occurs in ChatController rather than relying only on the text area's maximum length.

The account path begins with a SignupRequest or LoginRequest and ends with AuthResponse. Signup checks duplicate email, hashes the password with BCrypt and saves a User. Login retrieves the normalized address and verifies the supplied password against the hash. The response contains identity fields but no JWT or durable token. AuthContext stores the returned user data in local storage to support client presentation. These steps do not bind later message senders to an authenticated principal.

Account and room setup

3.4.1 Account and room setup

Signup validates a display name between 2 and 50 characters, a basic email pattern and a password of at least six characters. Email is trimmed and lowercased; duplicate email yields HTTP 409. A BCrypt hash is saved to the User document. Login returns unauthorized for an unknown email or mismatched password.

Room creation validates identifier length and allowed characters and returns conflict if the room exists. A room lookup returns the room or a not-found response. The frontend routes a successful room action to /chat/{roomId}.

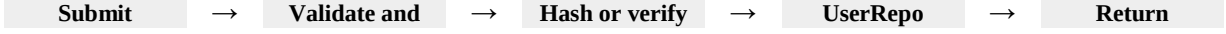




Fig 12. User registration and login workflow

Message handling

3.4.2 Message handling

A message request contains sender and content. The server rejects blank content, content over 2000 characters, a missing sender and a nonexistent room. The accepted content and sender are trimmed, the timestamp is assigned by the server, and the message is appended to the room before saving.

The STOMP method returns a Message and uses @SendTo to broadcast to /topic/room/{roomId}. History is exposed by a REST route that returns a slice in chronological order. Page size is clamped from 1 through 200.

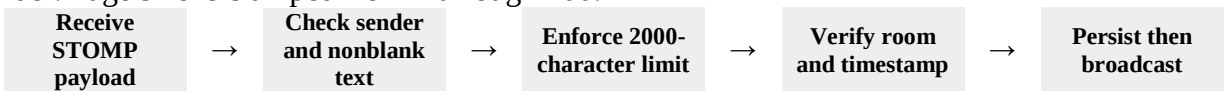


Fig 13. Message validation sequence

Presence and typing

3.4.3 Presence and typing

A join destination registers the current STOMP session. A leave destination removes it. The session disconnect listener also removes stale participant state when the transport ends. Participant events include room id, count, event type, username and user list.

Typing requests overwrite the event room id with the destination variable and are broadcast to the room typing topic. No database write occurs for typing. The client clears a typing badge after three seconds if a stop event is missed.

Interface behavior

3.4.4 Interface behavior

The composer is disabled while disconnected or while a send operation is pending. Empty content is ignored. Enter sends; Shift+Enter inserts a newline. A message input limit is set at 2000 characters and the UI shows a character count near the limit.

Chat history loading and participant loading begin together. A room-not-found error returns the user to the landing page. On message arrival the page updates its local list and, when the user is already near the bottom, scrolls to the newest content.

Important algorithms

History selection is performed against the room's embedded message list. For a request with page index p and effective size s, the service calculates an end position from the newest list boundary and a start position s elements earlier, with bounds adjusted so they remain within the list. The page-zero result is therefore the most recent slice rather than the earliest messages. The selected sublist is returned in chronological order so that the UI can render the conversation from older to newer. The requested size is clamped to a finite range, preventing an unbounded response array. The algorithm controls response length but does not avoid loading the full embedded Room document from MongoDB.

Presence uses three concurrent mappings to associate a WebSocket session with a room and user name and to find the sessions currently in each room. A join operation records the relationships; a leave or disconnect removes them. The service derives a participant list from the active room sessions and removes duplicate names before broadcasting a snapshot. Concurrent collections are appropriate for callbacks that can arrive from different connection threads, but they do not persist state or coordinate across application instances. A server restart therefore reconstructs no prior presence, and two backend processes would each know only about their own sessions without a shared presence store.

3.4.5 Important algorithms

History paging computes the end index as total messages minus page times page size, then selects the preceding page-sized range. This makes page zero the newest page and higher page indices older pages, with chronological order preserved within each result.

Presence updates use a session-to-room map, a session-to-user map and a room-to-session set. ConcurrentHashMap and concurrent key sets support simultaneous access. On removal, the session is deleted from each map and the room set is discarded when empty.

Message deduplication

3.4.6 Message deduplication

The frontend builds a message key from sender, timestamp and a content prefix. On initial load the keys populate a set. On live delivery, the handler first checks the set and then checks the current message array before appending. This protects the visible list from some overlaps between REST history and WebSocket events.

The method has a collision risk because two messages may share sender, timestamp and initial text. A unique message id generated by the backend is recommended for future refinement.

Error and connection handling

3.4.7 Error and connection handling

The frontend shows connecting/connected/disconnected status and guards the send path when the channel is not connected. API failures produce toast messages and inline room or history errors. The landing page handles common conflict and not-found responses with contextual feedback.

The inspected sources do not define a full centralized API error schema or end-to-end retry policy. Network reconnection behavior should be validated under disconnection and server restart conditions.

Implementation boundary

3.4.8 Implementation boundary

Implemented: signup/login endpoints with BCrypt, room creation and lookup, MongoDB persistence, message history pagination, STOMP room broadcasts, typing events, participant session tracking, disconnect cleanup and responsive chat interaction.

Not evidenced as implemented: JWT issuance, room authorization, message encryption, message editing/deletion, moderation, external broker, production deployment, automated load tests or a complete frontend test suite.

3.5 Logo Designing and Naming

The user-facing application is branded “PulseChat” in the React landing and chat interfaces. The name suggests conversational activity and is accompanied by a lightning icon from Lucide React. This branding is an interface label; the academic project title remains “Multi-user Chatroom Application.”

The inspected source includes decorative hero content and a chat preview, but no separate logo design source file or documented design process was supplied. The report therefore describes the visible application identity without claiming a formal branding study.

Visual identity in the interface

The interface uses a lightning symbol, room hash markers, initials-based avatars and status indicators to communicate chat concepts. The Avatar component derives initials and deterministic colors from a username. Such presentation choices are helpful cues, but they do not provide identity verification or security.

The report uses PulseChat only to refer to the application name visible in the source. The project title and student details on the cover remain those provided for this report.

3.6 Implementation Plan/Methodology

The implementation method follows an iterative integration sequence. First define account, room and message behavior; then configure data entities and REST endpoints; then configure the real-time messaging broker; then build the client workflow and connect each page to the backend. Finally review error states and document the verified behavior.

This sequence describes a reasonable reading of the system dependencies. The repository does not include dated planning records, so it should not be read as an exact historical schedule.

A reproducible local run uses the configuration in the project rather than relying on generic defaults. The backend properties set port 8081 and the MongoDB URI to localhost on the

standard database port with the chatApp database. The frontend Vite environment variables can override the HTTP API base and the SockJS URL. The allowed-origin lists include the documented local development origins. These values need to agree for both HTTP and messaging traffic.

Setup and execution

3.6.1 Setup and execution

Component	Setup requirement	Local command / address
Database	MongoDB 6+	Start local MongoDB on port 27017
Backend	JDK 21; Maven wrapper included	mvnw spring-boot:run; port 8081
Frontend	Node.js 18+ and npm	npm install; npm run dev; port 5173
Browser	Modern browser	Open http://localhost:5173

The README describes VITE_API_BASE_URL and VITE_WS_URL settings. Backend application.properties sets the MongoDB URI and server port. Ensure that the browser origin is allowed by both CORS and WebSocket configuration.

Validation approach

A useful validation plan separates checks by boundary. Account validation should cover field normalization, duplicate email handling, password encoding and login mismatch behavior. Room validation should cover accepted identifiers, rejected characters, length limits, conflicts and missing-room responses. History validation should cover an empty room, a short list, a list larger than the requested page, an out-of-range page and size clamping. These cases follow directly from controller and service branches and can be implemented as unit or integration tests without claiming that those tests have already been run.

The real-time path requires a different test harness because it includes a broker subscription and asynchronous events. A focused integration test can connect two clients, subscribe both to the same room topic, send one valid message and assert the event fields and persistence result. Separate cases should verify rejection of blank or oversized content and sending to a nonexistent room. Participant tests should exercise multiple sessions with the same display name, distinct names, explicit leave and unexpected disconnect. Typing tests should confirm that events are forwarded without creating a Message record and that the client-side indicator expires. The repository's existing context-load test does not provide these behavioral guarantees, so results for this proposed matrix must be reported only after execution.

3.6.2 Validation approach

A practical verification plan should cover account validation, duplicate email, invalid room ids, duplicate rooms, missing rooms, message length limits, empty messages, history page boundaries, join/leave events, disconnect cleanup and typing timeout behavior. The supplied repository only defines a Spring context-load test and does not record outcomes for this broader plan.

For academic accuracy, these cases are proposed validation checks. They must not be described as executed test results without a test log or observed evidence.

Limitations and challenges

3.6.3 Limitations and challenges

The principal engineering challenges are coordinating REST and WebSocket state, ensuring that message history and live delivery do not create duplicates, keeping presence consistent across disconnects, and handling concurrent sessions safely. The interface must remain understandable when the network is unavailable.

Limitations include process-local broker and presence state, embedded message arrays, lack of strong room authorization, no token-based session implementation, and limited automated testing evidence. These are concrete constraints derived from the implementation.

3.7 Source Code Structure and Module Responsibilities

3.7.1 Frontend entry points and routing

The browser application begins in main.jsx, which mounts the React tree and establishes the root rendering context. App.jsx supplies BrowserRouter, ToastProvider and AuthProvider before defining the route table. The root route renders LandingPage, while /chat/:roomId renders ChatPage. A catch-all route returns an unknown path to the landing screen. This structure keeps navigation concerns separate from individual page behavior and makes the two principal user journeys visible at the application boundary.

The route parameter is the room identifier used by the chat screen. The landing page navigates with a username in location state after a successful room operation. ChatPage also has a local-storage fallback so a refresh can recover the display name when the route state is absent. If neither source provides a username, the page returns the user to the landing route with feedback. This recovery is a client convenience and does not establish an authenticated server identity.

The frontend package is an ES module project driven by Vite. The package manifest declares React, React DOM, React Router, STOMP.js, SockJS client and Lucide React as runtime dependencies, with Vite and the React plugin in development dependencies. The manifest also includes TypeScript tooling, but the application pages and components inspected for this report are JSX files. The distinction matters when describing the delivered implementation: a declared tool is not evidence that the application was written in that language.

The route tree is intentionally small and communicates the prototype's application scope. There is no separate administrative route, room directory, profile page or moderation dashboard in the route table. Authentication is presented as part of the landing workflow rather than as a route guard. A future route expansion should preserve a clear boundary between view selection, account state and communication state, rather than allowing page components to become the only place where navigation policy is enforced.

3.7.2 Landing page and room access

LandingPage keeps the selected access mode, entered room identifier, guest display name, validation messages and loading flag in React state. The create and join operations share a validation function. For a guest, the display name must be present after trimming and must fit the frontend's two-to-thirty-character range. A room identifier must be at least three characters and match the letters, digits, underscore and hyphen pattern before a request is made.

The client can produce a suggested room identifier by selecting an adjective and noun from fixed arrays and appending a three-digit value. This helper improves convenience, but its output is not a security credential and should not be described as cryptographically unpredictable. The generated value is still passed through the same room validation as a user-entered value. The backend remains responsible for checking the final identifier and for returning a conflict if the room already exists.

Room creation calls the centralized roomApi and then navigates to the chat route when the REST request succeeds. A conflict response is translated into a warning and a field-level message that invites the user to join the existing room. Join behavior calls the room lookup endpoint; not-found and bad-request statuses produce a room-specific error. Other failures are presented through the toast system. A finally block clears the loading state in both success and failure paths. The page also coordinates account actions through AuthContext and exposes an authenticated user's name when one is available. A guest may continue with a display name without registration. The visual page contains decorative preview content, but the preview is illustrative rather than a live transcript. Documentation and demonstrations should distinguish that static preview from the ChatPage message list, which is populated from history and room events.

3.7.3 Chat screen and reusable components

ChatPage is the principal coordination component for room state. It obtains roomId from the router, obtains an initial username from navigation state or local storage, loads recent messages and participant names, connects the STOMP hook, and binds callbacks for message, typing and participant events. It also owns the input value, send state, typing users, message loading and

error state, current participant list, sidebar state and scroll behavior. This concentration reflects the needs of a single-room screen; a larger product could divide these responsibilities into dedicated hooks or state services.

MessageBubble is responsible for presenting one message and whether its sender matches the current display name. Avatar derives initials and a deterministic color from a name. ConnectionBadge translates the socket status into a compact visual cue. TypingIndicator presents active typers, and Toast provides non-blocking feedback shared across routes. CSS Modules isolate page and component selectors, while global.css and App.css provide shared styling and base rules.

The chat screen distinguishes loading, error, empty and populated states. When history is loading it displays a loading state; when the request fails it displays an error and retry control; when the resulting message array is empty it explains that the room has no messages; otherwise it renders a message list. The typing indicator is placed alongside the conversation rather than stored among persisted messages. This separation prevents a temporary activity cue from appearing later as historical conversation content.

Scroll behavior uses a message container reference, a bottom sentinel and state that records whether the user has moved away from the latest message. New events can be displayed without forcibly taking control away from someone reading older content. A separate control returns the user to the bottom. These choices are interface behaviors rather than guarantees about message ordering or delivery. They depend on the browser receiving events and the client updating its local state correctly.

3.7.4 Backend packages and responsibilities

ChatAppBackendApplication is the Spring Boot entry point. Configuration classes establish cross-origin HTTP behavior, the SockJS/STOMP endpoint, broker destinations and a handshake interceptor. Controllers are separated by interaction type: AuthController handles account requests, RoomController handles room operations and history, and ChatController handles room messaging and transient events. This division helps readers locate an operation by protocol instead of treating the backend as one large controller.

The DTO package contains transport shapes such as SignupRequest, LoginRequest, AuthResponse, MessageRequest, TypingEvent and ParticipantEvent. These objects define fields crossing the HTTP or messaging boundary and are distinct from MongoDB domain entities. The entity package contains User, Room and Message. Repositories provide persistence access, and RoomParticipantService owns in-memory session relationships. WebSocketEventListener removes participant state when a session disconnects.

Lombok annotations reduce repetitive constructors and accessors for several data classes. The Maven project declares Java 21 and Spring Boot 3.4.2, together with web, WebSocket, MongoDB, security-crypto and test dependencies. BCrypt is used from the crypto dependency, but the dependency declaration alone is not a full Spring Security authentication configuration. There is no configured authorization filter chain in the inspected application classes described by this report.

Package boundaries also reveal which concerns are not present as independent modules. There is no dedicated service layer for account workflows in the inspected code; AuthController performs validation, repository lookup, hashing and response construction. RoomParticipantService is a dedicated service because session state is shared across event handlers and REST reads. These boundaries may be adequate for a learning prototype, while future growth could move account and room policies into services with focused tests.

3.8 REST API Design and Request Handling

3.8.1 Request conventions and API utility

The frontend's roomApi module centralizes requests to the configured API base URL. Its request function attaches a JSON content type by default, calls fetch, reads the response as text, and attempts to parse that text as JSON. A non-success response becomes an ApiError carrying a readable message, numeric status and parsed response data. Network-level TypeErrors become a

separate connection message. This design gives the page components a common error shape without repeating low-level fetch handling.
The authentication API has a parallel helper in authApi.js. It posts a JSON payload, reads the response body and constructs AuthError when the response status is not successful. Keeping account and room clients in separate files reflects their separate endpoint groups, while preserving consistent status-aware behavior. The helpers use the Vite environment value VITE_API_BASE_URL when supplied and otherwise use the local backend URL at port 8081.
For a client request, the endpoint path and request method define the operation; JSON fields define the data being submitted. The server's response status distinguishes successful creation, successful lookup, invalid input, missing records, conflicts and failed authentication. The frontend branches on selected status codes to provide specific guidance. Other server errors are displayed using the response message when one is available, so a readable error body is part of the user-facing contract.
The API should be understood as a local prototype contract rather than a versioned external service guarantee. The prefix /api/v1 makes a version boundary visible, but the repository does not provide an OpenAPI specification or generated client. Contract tests would be useful before changing field names, status behavior or pagination defaults. A future specification should be generated or verified against controller behavior, rather than being maintained as an unrelated handwritten description.

3.8.2 Registration and login behavior

Signup is handled at POST /api/v1/auth/signup. AuthController trims the display name and email, converts email to lowercase and leaves the submitted password as a secret value for validation and hashing. The display name is required and must contain two through fifty characters. The email must match the controller's configured regular expression. The password must contain at least six characters. Invalid fields produce a bad-request response with an error message.
Before saving a new account, the controller asks UserRepo whether the normalized email already exists. A duplicate returns HTTP 409. Otherwise, BCryptPasswordEncoder encodes the password, and the new User is saved. The response is an AuthResponse containing the generated identifier, name and email. The response intentionally does not return the password hash. The unique index declared on User.email provides a database-level uniqueness constraint in addition to the pre-save existence check.
Login is handled at POST /api/v1/auth/login. The submitted email is trimmed and lowercased, and both email and password are required. UserRepo looks up the normalized email. An absent account and a password mismatch share an unauthorized response so the endpoint does not reveal which value was incorrect. A successful check returns the same identity fields used by signup. The inspected controller does not issue a token, cookie or signed session credential.
AuthContext stores the successful response in client state, writes the user object to local storage and separately persists the display name used by the chat route. A logout clears those local keys. This persistence lets the browser recover the display identity across a refresh, but it is not proof of identity to the backend. The current chat message payload includes a sender name supplied by the client. Any description of registration should therefore distinguish account verification and password hashing from enforced authorization on room access or message publication.

3.8.3 Room creation, lookup and participant endpoints

Room creation uses POST /api/v1/rooms and accepts JSON, plain text and a general application media type in its mapping. The controller extracts a roomId from a JSON-looking body or falls back to a plain string. It trims the result, rejects an empty value, enforces a length from three through fifty characters, and restricts characters to letters, digits, underscore and hyphen. Existing identifiers produce HTTP 409; a saved new room produces HTTP 201.
The JSON extraction routine is intentionally small. It searches for a roomId or room_id property and takes the quoted text following the colon. This is adequate for the frontend's simple JSON request, but it is not a full JSON parser and does not handle every escaped string or arbitrary

nesting case. A future contract should accept a typed request DTO and rely on the configured Jackson mapper, which would make parsing rules explicit and easier to validate.
Room lookup at GET /api/v1/rooms/{roomId} returns the matching Room document or HTTP 404 with an error body. The method is named joinRoom, but it verifies and returns a room; the actual live participant is registered later through the STOMP join destination. This distinction is useful when tracing the interface: an HTTP lookup is not the same event as registering a WebSocket session in RoomParticipantService.
The participant endpoint GET /api/v1/rooms/{roomId}/participants returns the current list from in-memory service state. A room with no tracked session returns an empty list. It does not create a room or retrieve a durable participant directory. Since this state is local to the running backend process, a restart clears it, and multiple backend instances would each observe only their own session mappings unless the architecture were changed.

3.8.4 History pagination and error interpretation

Message history is read from GET /api/v1/rooms/{roomId}/messages with optional page and size parameters. Defaults are page zero and size fifty. The server looks up the room first, returns not found when absent and returns an empty JSON array when the message list is empty. The requested size is clamped to at least one and at most two hundred, limiting an individual response even if a client requests an unreasonable page size.
Page zero is the newest segment. The controller computes an end index from the total message count minus page multiplied by page size, then computes a start index no smaller than zero. The selected sublist is returned in the stored order, which is chronological within that slice because messages are appended as accepted. Page one addresses an older segment. A page beyond the available range produces an empty list rather than an exception.
This approach operates on the room's embedded list after the Room document is loaded. Pagination limits the returned slice, but does not mean MongoDB fetched only that slice from a separate message collection. The full embedded list participates in document retrieval and in-memory slicing. For a small demonstration this is straightforward; for large rooms, a separate message collection and indexed query would provide a more scalable storage path.
The user interface requests the first fifty messages when a room opens. It also asks for participants concurrently, but treats a participant-list failure as an empty participant response so that a secondary status failure does not necessarily prevent history from loading. A history failure sets a visible error and produces a toast; an HTTP 404 also returns the user to the landing route. Additional pagination controls for older pages are a possible extension and are not claimed as an implemented interface feature.

3.9 Real-Time Messaging and Event Processing

3.9.1 Endpoint, transport and broker configuration

WebsocketConfig registers /chat as the STOMP endpoint and enables SockJS support. It allows the local development origins listed in the code: localhost and 127.0.0.1 on ports 3000 and 5173. The broker configuration enables the in-process simple broker for /topic and /queue, sets /app as the application destination prefix and /user as the user destination prefix. These values define how client destinations are routed through Spring's messaging layer.
The frontend hook constructs a STOMP client using the configured WebSocket URL and the SockJS factory. After connection, it subscribes to room messages at /topic/room/{roomId}, typing events at /topic/room/{roomId}/typing and participant updates at /topic/room/{roomId}/participants. This subscription set mirrors the three categories of data shown in the chat interface. Subscription callbacks parse JSON frames and forward events through component callbacks.
The application prefix directs client sends to controller methods. Message traffic uses /app/sendMessage/{roomId}; typing uses /app/typing/{roomId}; session registration and departure use /app/join/{roomId} and /app/leave/{roomId}. The server sends persisted chat messages back to a room topic through the method-level destination. Typing has its own topic, while participant events are published with SimpMessagingTemplate by the participant service.

SockJS is a transport compatibility layer around the endpoint configuration. The documentation should not imply that every session necessarily uses a raw WebSocket if the client library negotiates a fallback. Likewise, enabling a simple broker does not provide an external broker's durability or distributed routing. The configuration demonstrates a concise local event model and leaves broker relay, cross-node session state, connection authentication and production origin policy for later implementation.

3.9.2 Message send, validation, persistence and broadcast

When the user submits a message, ChatPage calls the send function returned by useStompClient. The hook checks that a client exists, is active and has completed connection before publishing. If those checks fail it returns false, allowing the UI to avoid treating the content as successfully sent. A successful publish serializes content, sender and roomId as JSON and targets the room-specific application destination.

ChatController validates the message body on receipt. Content must contain non-whitespace text and must not exceed two thousand characters. Sender must also contain text. The controller uses the room identifier from the STOMP destination as the authoritative room key, looks up that room and rejects a missing room. The payload's roomId is not relied upon to choose storage or broadcast routing, reducing inconsistencies between the route and body.

For an accepted message, the controller constructs a Message with trimmed content and sender and a LocalDateTime timestamp. It appends that object to the room's embedded list and saves the Room through RoomRepo. The method then returns the Message, and Spring's @SendTo directs the returned value to the matching room topic. The persistence operation therefore appears in the method before the broadcast return, so a successful event corresponds to a save attempt.

The flow is not a transactional delivery guarantee. The project does not define acknowledgements, durable broker delivery, idempotency keys or a unique message identifier. A sender could see a publish accepted by the client before the server response is observed, and reconnect behavior may leave uncertainty about whether a message was saved. The interface's deduplication key reduces some duplicate display cases but should not be described as a server-side exactly-once guarantee.

3.9.3 Join, participant update and disconnect lifecycle

The client announces a join after establishing subscriptions. The payload includes a sender name and room identifier, and the destination supplies the room key. ChatController obtains the WebSocket session ID from SimpMessageHeaderAccessor and registers the session when both the ID and display name are present. A single browser session is therefore associated with one room and one username in the in-memory service maps.

RoomParticipantService first removes an existing mapping for a session without broadcasting a departure, then records the new room and username and adds the session ID to a concurrent set for that room. It builds a participant event using the distinct, sorted usernames currently observed in the room. The event includes roomId, count, type, username and users, and is published to the room's participant topic. The visible count is based on the distinct username list, not on the raw session count.

When the client leaves explicitly, its hook attempts to publish a leave event before deactivating the STOMP client. The server removes the session mapping and broadcasts a departure if it had a known room and username. A WebSocket disconnect listener also removes the session if the transport closes unexpectedly. The cleanup path handles ordinary departures and unplanned disconnects, but the service is process-local and does not persist membership history.

ConcurrentHashMap and concurrent key sets permit multiple event threads to update shared session structures without using a single unsynchronized list. This is an appropriate local concurrency measure, but it does not make a cluster-wide presence service. The service has a session-to-room map, session-to-user map and room-to-session-set map that must remain mutually consistent. Focused tests should exercise add, repeated add, explicit removal, disconnect removal and removal of the last session in a room.

3.9.4 Typing events, deduplication and ordering

Typing state is intentionally transient. The frontend typing hook emits a true event the first time typing begins and resets a two-second inactivity timer on subsequent keystrokes. When the timer expires, it emits false. A user who explicitly stops typing also clears the timer and emits false if a typing interval was active. This limits repeated events while still communicating a useful start/stop signal.

The backend sets the event's roomId from the destination variable before returning it to the room typing topic. It does not save the event in MongoDB. On the receiving client, events from the local username are ignored. A true event updates the active-typing map and starts a three-second cleanup timer; a false event removes the sender immediately. The client timeout handles missed stop frames for display purposes, not as a server cleanup mechanism.

History and live events can overlap around connection time. ChatPage fetches the recent history and establishes the real-time connection through separate effects. It uses a message key composed of sender, time and the first forty characters of content to avoid adding an already observed item twice. It checks both a Set of seen keys and the current message array. This pragmatic key can collide for different messages that share these values, which is why a server-generated immutable identifier would be a stronger future design.

The server stores messages in insertion order and returns history slices chronologically, but the application does not define a global ordering policy across server nodes or retry attempts. LocalDateTime timestamps are created on the backend for persisted messages. Typing payloads, by contrast, include only sender, roomId and a boolean. If the system later introduces acknowledgements or multi-node delivery, it should define ordering, clock assumptions and duplicate handling explicitly rather than relying on arrival order alone.

3.10 Data Model and Persistence

3.10.1 User record and account lifecycle

User is mapped to the MongoDB users collection. It contains an identifier, display name, email, password field and createdAt timestamp. The email field carries a unique index annotation. Signup trims and lowercases the email before checking for an existing record and saving a new document. This normalization makes differently cased forms of the same address resolve to the same normalized lookup value in the controller's intended flow.

The password field stores the BCrypt encoded value produced by the controller. BCrypt is a one-way password hashing function with a configurable work factor, and the implementation uses the encoder's default configuration. Login compares the submitted secret with the stored hash using matches rather than decoding it. The response object contains identity fields and omits the password. The code does not encrypt or retain the submitted password as reversible data.

The createdAt value is initialized when the User object is constructed and is also initialized by its field declaration. The explicit constructor sets it to the current LocalDateTime. The project configures Jackson to serialize dates as readable values rather than numeric timestamps. This supports readable API responses, but the exact timezone and serialization policy should be documented before interoperating with clients in other regions.

Account existence should be kept separate from authorization. AuthContext caches the server response in local storage and exposes an isAuthenticated boolean based on the presence of a user object. Because the backend does not issue a session token and the inspected room and message endpoints do not validate an authenticated principal, that boolean is a client-side state indicator. A production identity model would need protected endpoints, token or session verification and sender identity derived from the authenticated principal.

3.10.2 Room and embedded message structure

Room is mapped to the rooms collection and has the MongoDB identifier, the externally selected roomId and a list of Message objects. A newly constructed Room initializes that list as an ArrayList. Messages are embedded records with sender, content and time. Saving a message currently means appending it to the room object and saving that document through RoomRepo.

Embedding supports a simple retrieval path: one room lookup supplies its messages, and the controller slices the list for a requested history page. It also ties message lifetime to room-document lifetime. The representation is simple to inspect and works for a bounded demonstration, but every save can involve a room document whose size grows with conversation history. MongoDB imposes document-size limits, and the application does not enforce a retention limit on the embedded list.

RoomRepo extends MongoRepository for Room and adds a finder for the logical roomId. The database identifier and roomId have distinct meanings: one is the stored document key; the other is the user-facing room name used in routes and message destinations. The repository interface delegates query generation to Spring Data. There is no explicit roomId unique index shown on the entity, so concurrent create requests would benefit from a database-enforced uniqueness rule rather than relying only on a preceding lookup.

A separate message collection would be a natural evolution if rooms become long-lived or message volume grows. That model could assign a stable message identifier, index by room and timestamp, paginate at the database and apply retention rules. Such a migration would require a backfill or compatibility plan for existing embedded arrays. The current report describes the present embedded model and treats the alternative as proposed work, not as an existing feature.

3.10.3 Session state and ephemeral data

Participant state does not reside in MongoDB. RoomParticipantService keeps session-to-room and session-to-username relationships and a room-to-session set in concurrent maps. It calculates a distinct username list on demand and publishes participant changes. These data structures represent the server's current view of connected sessions and are intentionally cleared when a session leaves or disconnects.

Typing state is also transient and is primarily maintained in client state. The server routes a TypingEvent to subscribers but does not save it. The client expires a typing display after a short timeout. This means a returning user does not see old typing indicators in history, which is appropriate because typing describes present activity. It also means the timeout should not be confused with a durable server-side typing-state cleanup job.

Persistent and ephemeral data have different reliability requirements. Accounts, rooms and messages should survive ordinary navigation and process restart when the database remains available. Presence and typing are meaningful only while connections are active. Keeping the categories separate simplifies the state model, but it also places importance on orderly disconnect processing and client reinitialization after reconnect.

If multiple application instances are introduced, an in-memory map on one instance will not know about sessions connected to another instance. A shared presence store or broker-supported user registry would then be needed, along with expiry for stale session entries. The current implementation should be evaluated as a single-process design. It should not be advertised as a distributed presence system solely because its collections are concurrent.

3.10.4 Data quality, validation and lifecycle

Validation happens at more than one boundary. The frontend checks fields before making a request to give immediate feedback, and the backend checks again before creating rooms, accounts or messages. Server-side checks are authoritative for the current request path because a client can be modified or bypassed. A future refactor could use Bean Validation annotations and typed request objects to reduce repeated conditions while retaining server enforcement.

The backend normalizes certain values intentionally. Signup and login normalize email case and surrounding whitespace. Room creation trims room identifiers and accepts only an allowlisted character set. Message handling trims content and sender after checking that they contain non-whitespace text. These transformations affect stored values, so they should remain documented as part of the contract and considered when designing future search or display behavior.

The project does not specify user deletion, room deletion, message editing, retention duration or data export. The absence of these lifecycle operations is a real boundary rather than an implicit promise that data is retained forever. A deployed service would need policies for account

removal, room ownership, backup duration and deletion propagation. It would also need to consider the implications of keeping messages embedded in a room document when an account is deleted.
Useful data-integrity checks include confirming email uniqueness, confirming that room lookups use the logical roomId consistently, verifying message timestamps and ordering, and testing that a missing room is not silently created by message submission. Tests should cover both expected valid data and boundary values such as an identifier at the minimum or maximum length, an empty list of messages and a history page past the end of the collection.

3.11 User Interaction and Client State

3.11.1 Account and guest identity flow

The user may interact with the landing page as a guest or use the account card to register or sign in. A successful account response is placed into AuthContext. The provider persists the returned user object under a local-storage key and separately saves the user's name for the chat page. The page uses the authenticated user's name when present; otherwise, it uses the trimmed guest name collected in the landing form.
The interface's authenticated state is computed from whether a user object exists in client state. It supports welcome text and account controls, but the backend does not issue a bearer token or establish a server-side security context in the inspected code. A browser user can alter local storage. Consequently, UI state is not a safe source of authorization decisions. The chat flow should be read as a prototype identity convenience, not as a protected account session.
Signup validation should be interpreted in two layers. The frontend offers immediate field-level guidance, while AuthController independently applies its own limits and email expression. The two layers are not identical in every detail: the frontend guest name limit is thirty characters, while the backend account name allows up to fifty. An account can therefore have a longer display name than the guest form currently permits. Such differences should be made intentional if requirements are later consolidated.
Logout clears the cached account and username keys and resets the landing mode. It does not delete the server-side User record or erase historical room messages. No account deletion route is shown. The distinction between signing out, leaving a room and deleting account data should remain clear in the UI and in future requirement definitions. These are different actions with different data effects and should not be represented by one generic exit control.

3.11.2 Room creation and joining sequence

A user chooses create or join mode, enters or generates an identifier and provides a guest name if the account context does not already supply one. The client trims the identifier and validates its characters before enabling a request. During the request, a loading state helps prevent the user from interpreting a pending operation as completed. A successful room response is followed by route navigation; the room screen then performs its own history and participant reads.
Creation and joining use different REST semantics. Creation submits a new identifier and is expected to fail if that identifier already exists. Joining verifies a known room by identifier and is expected to fail when no record matches. The room route itself is entered only after the respective REST operation succeeds. This reduces the chance of opening a chat view for a room the server cannot find, although a room may still be deleted or become unavailable between the initial lookup and later events in a future implementation.
Generated identifiers combine a randomly selected adjective, noun and three-digit suffix from small local lists. The resulting string is convenient to communicate, but the sample space is finite and the generator uses Math.random. It should not be considered an access-control secret. The backend's allowed identifier pattern permits manual values as well, so a room ID is a locator in the current prototype rather than a secure invitation token.
Feedback distinguishes common cases. A duplicate room prompts the user to join instead. A missing room gives guidance to check the identifier or create a room. A network failure is presented as an inability to reach the server. These messages support ordinary recovery, while detailed server diagnostics remain available in response objects or developer consoles. Future

work could add retry with bounded delay, explicit cancellation and a clearer distinction between server unavailable and request rejected.

3.11.3 Conversation loading and rendering

When ChatPage mounts for a room, it requests page zero of history with a page size of fifty and obtains participants in parallel. The participant request has a local catch that substitutes an empty list if that endpoint fails. History loading controls the main loading indicator. If history retrieval fails, the page records an error, shows a toast and offers a retry action; a 404 also sends the browser back to the landing view.

The history array is used to initialize both the message list and a Set of deduplication keys. Live message callbacks compute the same key and avoid adding a duplicate to the Set or array. This helps when a message delivered around initial connection is also present in the fetched history. The key uses sender, timestamp and a content prefix. It is a practical client heuristic and is not as collision-resistant as a stable ID assigned by the server.

MessageBubble receives each message and a boolean indicating whether its sender matches the current display name. It can therefore render sent and received messages differently without changing the stored Message shape. System-style items are handled through a separate render branch if an incoming object carries the system marker, but the server event model described in the current implementation should not be assumed to produce persistent join messages.

The message list uses a bottom reference and tracks user scrolling. It automatically follows new content when the user is already near the bottom, while preserving a scroll position for someone reading earlier messages. A button offers an explicit return to the newest message. This design is useful in live conversations, but the current report does not claim virtualization or efficient rendering for arbitrarily large lists. Pagination limits initial retrieval; future older-history navigation would need to integrate with the scroll anchor.

3.11.4 Composer, status and responsive behavior

The composer is tied to the connection state. The text area is disabled while disconnected or while a send is in progress, and the send button is disabled for blank trimmed content. The input limits its length to two thousand characters, matching the server-side message check. Enter sends a message, while Shift+Enter inserts a line break according to the interface hint. This makes the keyboard behavior visible rather than requiring users to infer it.

Typing callbacks are connected to input changes and the useTypingIndicator hook. The hook sends a start event once per typing interval and resets its inactivity timer as additional keys arrive. Explicit stop behavior clears the timeout. A receiver excludes its own name from the typing list and expires other users after three seconds. Since sender names are supplied by clients, the indicator is a presence cue rather than an authenticated attribution guarantee.

ConnectionBadge presents the states used by the STOMP hook, including connecting, connected, disconnected and error conditions. The hook updates React state on connection and disconnection callbacks, and it cleans up its subscriptions when the client disconnects. Network errors are logged to the console and change the visible status. The report does not claim that every transient outage automatically recovers without user action unless that behavior is explicitly observed in the implementation.

The sidebar can be opened or collapsed and contains the room identifier, current display identity, participant list, connection badge and leave action. CSS Modules are used for layout and component styling, while global styles establish common visual rules. Responsive CSS can rearrange the layout at narrower widths, but a full accessibility or device-compatibility evaluation has not been supplied. Keyboard navigation, screen-reader naming and contrast should be verified with a dedicated test plan.

3.12 Local Configuration and Reproducible Execution

3.12.1 Toolchain and local services

The backend pom.xml sets Java 21 and Spring Boot 3.4.2. The Maven Wrapper is included, so a separate system Maven installation is not required for the standard wrapper command. The frontend manifest uses Vite and declares React 19, React Router 7, STOMP.js, SockJS client and

Lucide React. Node.js and npm are needed to install frontend dependencies and run the development or build scripts.
MongoDB is configured through application.properties with a local URI at mongodb://localhost:27017/chatApp. The backend's configured server port is 8081. This source value should be treated as authoritative for the checked-out implementation. Some README snippets describe a different port or list technology versions that do not agree exactly with the current package manifest; when documentation conflicts with executable configuration, verify the source and update the README before relying on it.
A local run requires three cooperating processes: the database service, the Spring Boot backend and the Vite frontend. The database should be reachable before the backend begins repository operations. The backend should be listening on the configured port before the frontend sends HTTP requests or opens the SockJS endpoint. The frontend development server is normally accessed at localhost:5173, subject to Vite configuration and any custom environment settings.
The package lock file records a resolved frontend dependency tree and should be used with npm install or npm ci according to the intended workflow. Reproducible builds should retain the lock file, JDK version and Node version used for verification. Environment-specific service URLs should be supplied through frontend environment variables rather than committed secrets. The repository contains sample configuration information, but no evidence of a production secrets manager or external hosting target.

3.12.2 Start-up sequence and readiness checks

Start MongoDB using the locally installed service or the database process appropriate for the operating system. Confirm that the configured URI can be reached before diagnosing application-level failures. The project does not include a dedicated readiness dashboard in the inspected code. A database shell connection or service status command can provide an environmental check, but such a check should be recorded separately from a test of the chat application itself.
From the backend project folder, the wrapper can run the Spring Boot application. The console output should indicate that the application context started and that the web server is listening on the configured port. If startup fails, inspect the first relevant exception for database connection, port conflict, dependency resolution or Java version causes. Logging levels in application.properties are configured for WebSocket and application packages and can assist local diagnosis.
From the frontend folder, install dependencies and run the Vite development server. Open the URL printed by Vite rather than assuming a custom port. The frontend API default points at the backend's local 8081 URL. The STOMP client similarly needs the backend's /chat endpoint. If environment files override these values, verify that both URLs target the same active backend and that the allowed origins include the browser origin.
A useful readiness walkthrough is to load the landing page, create a new room, open the room and observe the connection badge change, then send a short message. Verify that the message appears in the sender's interface and, using a second browser session, in another subscribed client. Reloading the room can demonstrate retrieval from the history endpoint. This walkthrough is an operational procedure; it is not a recorded acceptance result unless someone executes it and documents the outcome.

3.12.3 Configuration consistency and common failures

The backend property server.port is 8081, while older README text in the project describes port 8080 in several examples. A mismatched frontend VITE_API_BASE_URL or VITE_WS_URL can therefore make a correctly running backend appear unavailable. The configuration should be reconciled before a demo, and a single source of truth should be selected for local development. The same check applies when the frontend runs on a port other than the origin allowlisted by CORS and WebSocket configuration.
A MongoDB startup error often indicates that the service is stopped, the URI is incorrect or the local port is occupied. A room-not-found response indicates that the identifier does not resolve to an existing Room document; room IDs are case-sensitive according to the string lookup behavior.

A duplicate-creation response is a conflict rather than a transport failure, and the client intentionally suggests joining the existing room.
A stuck connecting badge can arise from an unavailable backend, an incorrect WebSocket URL, an origin mismatch or a failed SockJS handshake. Inspect the browser console and backend logs together so the client symptom can be correlated with server evidence. A test made from localhost may pass while a deployment from a different origin fails, because the allowed origins are explicitly enumerated in the configuration.
Port changes require coordinated edits. Updating server.port alone is not sufficient if the frontend environment variables still point to the old HTTP and WebSocket endpoints. Updating the Vite port may also require corresponding CORS and WebSocket origin changes. Configuration review should treat these values as a connected set and verify the final browser origin, backend address, database URI and broker endpoint after the change.

3.12.4 Build and handover procedure

The Maven Wrapper provides a consistent backend command entry point. A handover note should list the JDK version, wrapper command, required local MongoDB service and test command. The context-load test uses a test URI and mocks its repositories and messaging template; it should be described as a wiring test. If a full test command is run, preserve its actual console result and do not substitute the README's historical output without confirming it against the current environment.
The frontend package provides development, build and preview scripts. The build output is written to the dist directory by Vite. A successful build demonstrates that the current source can be bundled, but it does not verify backend integration or browser behavior. A preview server serves the built assets and should use the same environment configuration assumptions as the intended target. The committed dist directory, if present, should not be confused with a deployment artifact that has been validated for a production host.
A repeatable handover includes project folder paths, required environment variables, local service ports, startup order, build steps and known limitations. Configuration values that differ between a developer machine and another environment should be kept out of source control when they are secrets or machine-specific. No database password or secret is needed by the current local URI, but a hosted database would require secure credential handling.
The delivered application is best reproduced as a local three-part system rather than as a single executable. A new developer should first establish compatible Java, Node and MongoDB versions, then install frontend packages, start MongoDB, launch the backend and finally start the Vite client. Recording the date, dependency lock state and exact commands makes later comparisons more meaningful and helps distinguish an application regression from environmental drift.

3.13 Requirements Traceability and Technical Terminology

3.13.1 Traceability from user need to implementation

The need for synchronous conversation maps to the STOMP room topic and the ChatController message handler. The need to revisit recent discussion maps to the MongoDB Room document and the paginated messages endpoint. The need to understand who is currently present maps to RoomParticipantService and the participant topic. The need for short-lived activity feedback maps to the typing destination, TypingEvent and frontend expiry timers.
The need for a clear room context maps to room identifiers in the route, header and sidebar. The need for connection awareness maps to useStompClient status callbacks and ConnectionBadge. The need to reduce repeated names and presentation code maps to reusable Avatar, MessageBubble, Toast and typing components. Each traceability link points to inspected source behavior; it is not a statement that the requirement has been validated under every browser or network condition.
Some requirements remain partially satisfied. Account creation and password verification exist, yet the authenticated principal is not enforced on room APIs or message destinations. History retrieval exists, yet older-history navigation is not shown as a user feature. Participant updates

exist in a single process, yet distributed presence is not demonstrated. These partial links should be labeled as bounded or incomplete rather than counted as fully delivered production requirements.

A future requirements register can assign each requirement an identifier, priority, source, implementation file, verification case and status. Status values should distinguish implemented, partially implemented, proposed and verified. For example, a source file can establish that validation code exists, while only a test result can establish that a particular boundary case passed in a particular environment. This distinction makes traceability useful to both developers and evaluators.

3.13.2 Glossary of communication and data terms

A WebSocket is a long-lived, bidirectional communication channel between browser and server. STOMP is a messaging protocol that represents operations as frames and uses destinations for send and subscribe behavior. SockJS supplies a client/server transport library configured here around the /chat endpoint. REST refers to the HTTP request/response interface used for account, room and history operations.

A topic is a broker destination to which multiple clients can subscribe. A room topic is scoped by a room identifier so subscribed clients receive events associated with that room. A DTO is a data transfer object used to shape request or response payloads. A repository is the Spring Data abstraction through which the controllers and application code request persistence operations.

A persistent record is saved in MongoDB and can be read after a normal page reload when the database remains available. An ephemeral event represents short-lived state such as typing or current online presence and is not written to the message history. A session ID identifies one WebSocket connection in the server's in-memory participant tracking. It is distinct from a user account identifier.

A BCrypt hash is a one-way representation used to verify a password without storing the original secret. Authentication establishes or checks a user's identity; authorization decides what an identified user may do. The current project includes account verification behavior but does not demonstrate complete authorization enforcement for room access and messaging. This glossary preserves that distinction throughout the report.

3.13.3 Glossary of implementation and verification terms

A React component is a reusable unit of user-interface rendering and interaction. A hook encapsulates stateful behavior that can be reused by components; useStompClient manages the messaging client lifecycle and useTypingIndicator manages typing event timing. A CSS Module scopes class names to a component or page stylesheet, while global CSS applies shared rules across the application.

Pagination divides a large logical result into bounded slices. The current message endpoint chooses a recent slice from the room's embedded list and returns that slice chronologically. Deduplication attempts to avoid displaying the same observed message more than once. The current client uses a composite key, so a stable server-generated identifier would be a stronger foundation for future reconciliation.

A unit test focuses on a small behavior, often with collaborators replaced by test doubles. An integration test verifies multiple components working together, possibly with a real database or messaging broker. A context-load test checks that the Spring application can assemble its beans, but it does not by itself verify endpoint behavior or end-to-end delivery.

A limitation is a known boundary of the current implementation. A future enhancement is a proposed change, not a completed feature. A measured result requires an actual execution and recorded method. This vocabulary helps prevent planned tests, recommended security controls and possible deployment options from being mistaken for functionality already present in the supplied code.

3.14 Worked REST API Exchanges and Contract Compatibility

3.14.1 Registration and login request examples

A registration request is a JSON object sent with POST /api/v1/auth/signup. Its fields are name, email and password, corresponding to SignupRequest. A representative local example might use a display name such as Mira Das, an email address such as mira@example.test and a development-only password. The example values are illustrative and should not be copied into a live account. The controller trims the name and email, converts the email to lowercase and validates the submitted password before storage.

A successful signup returns HTTP 201 and an AuthResponse containing the saved identifier, name and normalized email. The response is not the User document's password field and should not expose the encoded password. A repeated email returns HTTP 409 with an error field. Invalid name, email or password values return HTTP 400 with an error field. These outcomes provide predictable branches for the landing page and for a future API client.

A login request uses POST /api/v1/auth/login with email and password fields. The controller normalizes the email and checks that both values contain text. An unknown email or a mismatch against the stored BCrypt value returns HTTP 401 with a common invalid-credentials message. A successful login returns HTTP 200 with the same identity response shape as signup. This response does not contain a token or a server-issued session identifier.

When documenting an example, distinguish the request from the result. The endpoint behavior is established by controller code, but a sample response with a particular MongoDB ID or created timestamp would be invented unless an execution trace were captured. For an academic example, show the field names and status code and label any concrete values as illustrative. A test should assert the shape without depending on an arbitrary generated identifier.

3.14.2 Room creation and lookup exchanges

A room creation request is submitted to POST /api/v1/rooms. The frontend sends JSON containing roomId, such as a development identifier with letters and hyphens. The backend extracts the identifier, trims it and applies a three-to-fifty-character range and an allowlist of letters, digits, underscore and hyphen. The frontend checks a similar pattern before sending, but server validation remains authoritative because requests can originate from outside the supplied interface.

For a valid unused identifier, the controller creates a Room and returns HTTP 201 with the saved room object. If the identifier is already present, it returns HTTP 409. If the input is missing, too short, too long or contains unsupported characters, it returns HTTP 400. A client should treat these statuses as different conditions: a conflict suggests joining an existing room, while malformed input should be corrected before retrying.

Room lookup uses GET /api/v1/rooms/{roomId}. A found room is returned with HTTP 200, and a missing room returns HTTP 404 with an error field. The returned object can contain the embedded message array, so clients should use the dedicated messages endpoint when they need a bounded history slice. The current frontend's join helper checks room existence; the chat page later loads messages and registers the live connection.

The backend accepts more than one content type for creation and has a fallback for plain text, but the supplied React client uses JSON. Supporting multiple forms can be useful for compatibility, yet it expands the parsing cases that need tests. A typed request object would offer a clearer contract and delegate JSON parsing to Spring. If compatibility requires plain text, that behavior should have an explicit test and be documented as a deliberate alternative.

3.14.3 Message history and participant exchanges

A history request is a GET request to /api/v1/rooms/{roomId}/messages with page and size query parameters. The browser currently requests page zero and size fifty. The server returns a JSON array of Message objects containing sender, content and time. If the room exists but has no messages, the result is an empty array. If the room itself is not found, the result is HTTP 404 rather than an empty history.

The server clamps a requested size to the interval from one to two hundred. Page zero selects the newest slice, and the items inside the slice remain chronological. A later page selects an older slice. Because the source loads the Room document and slices the embedded list in application memory, a successful bounded response should not be described as database-level pagination. The request controls the returned array size, not necessarily the amount of stored data read.

The participant request uses GET /api/v1/rooms/{roomId}/participants and returns a JSON list of distinct usernames currently found in the process-local session maps. A room with no sessions yields an empty list. The endpoint does not return account profiles or a historical list. Since it reads volatile state, the result can change immediately after a join or disconnect event.

A client should handle an empty participant list separately from an unavailable endpoint. The current ChatPage requests history and participants together, with a fallback to an empty participant list if that secondary request fails. This allows the main conversation load to proceed in some partial-failure cases. A future API response could include stronger freshness metadata, but the current interface presents only the current list and count supplied by the service.

3.14.4 Error response handling and client mapping

RoomApi and AuthApi read the response body as text and attempt JSON parsing. When parsing succeeds and the object contains error, that field becomes the Error message; otherwise, the code falls back to the string body or an HTTP status label. ApiError and AuthError preserve status and parsed data so a component can choose a status-specific response. A fetch TypeError is translated into a local server-connection message.

The landing page uses status-aware handling for the common room cases. A create conflict produces a warning and encourages joining the room. A not-found or bad-request join result becomes a room-not-found prompt. Other errors are sent to the toast system. Registration and login cards can show endpoint validation feedback. These behaviors prevent a raw stack trace from appearing in the interface while still retaining useful response information for debugging.

Error text should not become a substitute for protocol status. An HTTP 409 for a duplicate room is different from a network exception where no response arrived. An HTTP 401 for invalid credentials is different from a backend being unavailable. Keeping those categories separate improves recovery guidance and makes automated tests less dependent on exact prose. Error messages may evolve, whereas status semantics should be stable and documented.

Future error contracts could define a consistent object with code, message and optional field errors. The current controllers use simple maps for many error responses and rely on Spring's default handling for some thrown messaging exceptions. A centralized exception handler could produce consistent JSON for validation, missing records and persistence failures. Any such change would need matching client tests and should preserve the existing useful status distinctions.

3.14.5 API compatibility and contract maintenance

The /api/v1 prefix communicates an intended version grouping, but no formal schema document or generated client appears in the inspected project. The frontend API helpers contain endpoint paths and request shapes in JavaScript. The Java controller annotations and DTO fields define the corresponding backend contract. These two sources should be reviewed together when changing a route, a field name or a response shape.

A change to MessageRequest affects the frontend publisher, ChatController and potentially integration tests. A change to Message affects the broadcast payload, the history response and MessageBubble. A change to ParticipantEvent affects RoomParticipantService and the participant callback. Listing these cross-layer dependencies before an edit reduces the risk of a change that compiles on one side but leaves the browser reading a missing property.

Compatibility tests can serialize representative DTOs, send requests to controller endpoints and validate JSON properties. For REST, contract cases should include status and body fields. For STOMP, tests should verify destination strings and event payload fields. A build succeeding is not sufficient evidence that two independently developed layers still agree on semantics such as room ID authority or timestamp formatting.

When an API change is not backward compatible, incrementing a version path or supporting a transition period may be appropriate. For this small project, versioning can remain lightweight, but a written decision record should explain whether old clients are supported. The important principle is to treat protocol shape as a user-facing interface with explicit evolution rules rather than as an incidental consequence of Java field names.

3.15 Source-to-Behavior Maintenance

3.15.1 Changing room validation safely

Room identifier behavior appears in both LandingPage validation and RoomController.createRoom. The frontend pattern currently permits letters, digits, hyphens and underscores and enforces a minimum length. The backend trims and validates the final request with its own length and character rules. A requirement change should update both layers, associated help text and tests so the browser does not reject values the backend accepts or promise values the backend rejects.

The room controller's extractRoomId method supports JSON property names roomId and room_id through manual string search and also strips quotation marks from a plain body. This implementation is compact but has edge cases around escaped quotes and malformed nested JSON. Changing the accepted room format is an opportunity to move toward a DTO with Bean Validation, while preserving a compatibility path only if an actual client depends on the plain-text form.

Room uniqueness is currently checked by a repository lookup before saving. Under concurrent create requests, two requests could both pass the lookup before either save completes unless storage also prevents duplicates. A unique index on logical roomId would enforce the rule at the database boundary. If added, the duplicate-key exception should be translated to the same conflict status so callers retain a stable contract.

After a validation change, verify the landing page error text, backend response status, saved value normalization and join lookup behavior. A unit test for the predicate is not enough if routing and persistence also depend on the normalized identifier. Document whether room IDs are case-sensitive and whether normalization occurs; current code trims but does not lowercase room IDs.

3.15.2 Changing the message model

The current Message object has sender, content and LocalDateTime time. It is embedded in Room.messages, is returned by history and is broadcast by the STOMP handler. Adding a field therefore affects Java serialization, existing MongoDB records, frontend message rendering, deduplication and any example payloads. Before adding fields, decide whether older documents can omit the field and what default behavior the client should use.

A server-generated message ID would provide a stable identity for React keys and duplicate detection. It could also support edit or delete operations later, although those actions require authorization and audit decisions. Introduce the ID in a backward-compatible way, make the client tolerate old records without it during migration and replace the composite key only after both persisted and live paths carry the new field.

A timezone-aware timestamp is another possible evolution. LocalDateTime has no timezone offset and can be ambiguous when clients interpret it outside the server's local context. An Instant or offset-aware representation with a documented serialization format would improve cross-region interoperability. Existing data migration and sorting logic should be considered before changing the Java type.

Moving messages to their own collection changes persistence semantics. The controller would no longer load, mutate and save the entire room to append one item. A repository query would need to select by room ID, sort by timestamp or sequence, and page from the database. This change should include indexes, retention policy, transaction or ordering decisions and migration tests rather than only a modified entity class.

3.15.3 Changing the participant event contract

ParticipantEvent is broadcast by RoomParticipantService and consumed by ChatPage. Its fields are roomId, count, type, username and users. The client uses count when present and prefers a

nonempty full users list; otherwise, it applies a JOIN or LEAVE event to its local list. A contract change must coordinate the producer and consumer, including the behavior when users is intentionally empty.

The current count comes from the distinct usernames list included in the event, whereas RoomParticipantService also exposes getCount based on the session set. These values can differ when multiple sessions use the same name. A future contract should decide whether count means sessions, accounts or distinct display names and use that definition consistently in the service, REST endpoint and interface.

If account identities become trusted, a participant event should use a stable user identifier internally and a display name for presentation. Names are mutable and not unique. A browser may also intentionally connect more than once. Separating connection count from distinct member count makes the interface's semantics clearer and can support a separate active-session indicator if needed.

Tests should assert both the event payload and client reconciliation. Verify join, leave, duplicate display name, empty room, disconnect and stale event behavior. Versioning may be warranted if a new payload shape is introduced while clients can be cached. During transition, the backend can publish a compatible shape or the client can accept both forms, provided the compatibility logic is covered and later removed deliberately.

3.15.4 Adding an API route or user-interface feature

A new REST feature should be represented across a controller mapping, a request or response DTO where appropriate, a service or repository operation, a frontend API helper and a UI state transition. Define validation, success status, error status, authorization assumptions and persistence effects before coding. A route should not be added only to make a button appear functional; its server behavior and client failure path are part of the feature.

A new real-time event should define its destination, payload schema, sender, recipient scope, persistence policy and reconnect behavior. Add a controller handler, a topic publication path and a frontend subscription callback. Ensure the event is distinct from persisted messages if it is ephemeral. Test that a room subscriber receives it and that an unrelated room does not.

A new reusable UI component should be evaluated for props, empty state, loading state, error feedback, keyboard behavior and CSS scope. Keep shared concerns such as API calls, authentication context, toasts and WebSocket subscriptions in their existing modules unless a refactor has a clear benefit. Include a browser test for the complete user journey, not just a component snapshot.

The change record should identify altered files, user-visible behavior, new configuration, database effects and verification evidence. Update README instructions only after checking them against application.properties and package.json. This source-to-behavior map makes maintenance more reliable than editing the nearest screen while overlooking the server, transport or data model dependencies.

3.15.5 Review and release checklist

A code review should trace the complete data path and check untrusted inputs at the backend boundary. For account work, inspect normalization, password handling and authorization consequences. For room work, inspect identifier validation and uniqueness. For message work, inspect sender authority, content bounds, persistence-before-broadcast semantics, event destinations and client deduplication.

A release candidate should be built from a known source revision with the Java and Node toolchains recorded. The backend test command and frontend build should be run, and any manual browser test should have a saved evidence record. Configuration should be checked for correct API base URL, WebSocket endpoint, server port, allowed origin and database URI. Secrets should not be committed in environment files.

A deployment review should ask whether TLS is enabled, whether a real identity is enforced, whether rooms are private or public, whether rate limits exist, where logs are stored, how backups are restored and what happens when MongoDB is unavailable. These are gate questions

for production deployment; they are not claims that the prototype currently implements those controls.

After release, monitor connection failures, database exceptions and message processing errors. Keep a rollback path and a way to compare the deployed configuration with the tested configuration. The current repository does not include an operational monitoring system, so any such process would need to be designed and documented before a public deployment is described as production-ready.

3.16 Data Fields, Payloads and Validation

3.16.1 Account request and response fields

SignupRequest has name, email and password fields. Name represents a user-facing label, email is normalized before repository lookup, and password is the submitted secret used to create a BCrypt hash. The DTO is a transport structure and is not the User document itself. A response uses AuthResponse with id, name and email. The identifier is generated by persistence, while the response intentionally excludes password material.

LoginRequest carries email and password. The backend trims and lowercases the email, requires both fields to contain text, looks up the user and compares the submitted password with the stored encoded value. The returned AuthResponse does not include a token. This field catalogue should be consulted when designing a client form or test fixture, but it should not be interpreted as a complete secure session schema.

The User document adds fields not present in the signup response: a database identifier, encoded password and createdAt timestamp. Email has a unique index annotation. The controller constructs the User after validation, and the entity constructor initializes createdAt. A future migration should document whether email changes are supported, how uniqueness is maintained and how account deletion relates to room messages.

Data handling should minimize exposure. Password values should not be logged, echoed into error bodies or placed in local storage. The current AuthContext stores the response identity but not the submitted password. A security review should inspect logs, browser state and response payloads to verify that no unintended secret copy is introduced by future diagnostic code.

3.16.2 Room, message and history fields

Room contains an internal id, a logical roomId and a list of messages. The logical identifier appears in HTTP paths and STOMP destinations; it is not the same field as the MongoDB document id. The entity initializes messages as an ArrayList. RoomRepo finds a room by the logical roomId and otherwise inherits standard MongoRepository operations keyed by the database id.

Message contains sender, content and time. ChatController trims sender and content and creates the time on the server. The object has no stable message ID in the current source. The browser therefore derives a composite key for deduplication. If adding a stable identifier, update the entity, persisted documents, WebSocket representation, REST history, UI list keys and migration behavior together.

MessageRequest carries content, sender and roomId. The handler uses the destination room identifier as authoritative rather than trusting a different roomId in the body. Sender remains client-supplied and is checked only for nonblank content. The type has a room field for request context, but the backend destination determines which room is resolved for the operation.

History query parameters are page and size. The default page is zero and the default size is fifty; the server clamps size to one through two hundred. Page zero selects the most recent slice. A returned array is chronological within the slice. A future cursor-based API could avoid offset recomputation and work more naturally with an independent message collection, but it would require a new cursor contract and tests.

3.16.3 Transient event fields and meanings

TypingEvent contains sender, roomId and a boolean typing field. The destination room ID is copied onto the event by the server. A true value indicates that the client currently considers the sender active in the composer; false indicates the sender has stopped. The event is sent to a room

topic and not persisted. The UI also applies a local expiration timer to avoid indefinite stale display.
ParticipantEvent contains roomId, count, type, username and users. Type is used for join and leave conditions. Users is a list of distinct sorted display names assembled from current session mappings. The count published by the service is the size of this list. This means the event's count represents distinct names in that publication, not necessarily active socket sessions or authenticated account records.
The application sends join and leave through MessageRequest-shaped payloads containing sender and roomId, but the server obtains the room from the destination and uses the session ID from the message header to update membership. The room value in the body is contextual rather than authoritative for the map. Sender values remain untrusted display labels until tied to a server-authenticated identity.
A payload contract should state which fields are required, who sets them, whether they are persisted, and which destination receives the event. It should also document null handling and unknown fields. DTO constructors and Lombok accessors make the objects convenient Java payloads, but clients should rely on a documented JSON contract rather than the incidental order of Java class fields.

3.16.4 Validation and normalization map

Account names are trimmed and checked for two to fifty characters at the backend. The guest interface currently checks two to thirty characters. Email is trimmed, lowercased and checked against a configured regular expression. Password is required and has a six-character minimum; the code does not demonstrate a complexity policy, password reset flow or rate limiting.
Room identifiers are trimmed, must be between three and fifty characters and must match an allowed character set of letters, digits, underscore and hyphen. Room identifiers are not lowercased by the controller, so case remains meaningful to repository lookup. The frontend applies a similar pattern before sending, and the server repeats validation to protect the persistence boundary.
Message content and sender must contain non-whitespace text. Content has a maximum length of two thousand characters. Accepted values are trimmed before the Message is constructed. These checks do not perform HTML sanitization, spam detection or moderation policy. Because rendered messages should be treated as user-supplied content, a future browser security review should inspect React rendering and any change that introduces raw HTML.
A centralized validation specification could express field, normalization rule, boundary, error status and test identifier in one table. This would make cross-layer differences visible, especially the guest/account name length difference. The existing report describes the actual rules above; proposed consolidation should not be represented as if it already exists.

3.16.5 Serialization and compatibility notes

Spring's configured Jackson mapper serializes LocalDateTime values as readable date strings rather than numeric timestamps. The exact representation should be verified with an integration test before another client depends on it. LocalDateTime does not carry an offset, so clients in different time zones may need a clearer UTC or offset-aware contract in a future revision.
The frontend's request helpers parse response text as JSON when possible and otherwise preserve the text. This accommodates JSON error maps and simple string errors. When extending response types, maintain stable property names and status semantics. If the payload changes, tests should cover both backend serialization and frontend parsing rather than only compiling the Java or JSX files.
Older MongoDB documents may lack fields added later. Entity constructors and default values can help when creating new objects, but they do not automatically migrate every historical document. A migration should examine existing collection data, define defaults and preserve recoverability. No migration framework or production migration procedure is included in the supplied project.

Unknown fields and optional values should be handled deliberately. A UI component should tolerate an absent email or a missing optional participant list without breaking the room. A server should reject required values that are absent and return a consistent error. Compatibility tests make those choices explicit and reduce accidental dependence on serializer defaults.

CHAPTER 4
RESULT ANALYSIS AND VALIDATION

4.1 Implementation of Solution

The implemented system is a browser-based chat application composed of a React frontend, a Spring Boot backend and MongoDB. Its user workflow covers account access, room creation or joining, message history, live message delivery, presence feedback and room exit. REST and STOMP are used for distinct interaction types.

The implementation evidence consists of source files, package manifests, Maven configuration, application properties and the single context-load test definition. These sources establish technologies and control flow, but they do not establish measured latency, concurrent-user capacity, a completed user study or a public deployment. The report therefore separates code-observed behavior from test plans and recommendations.

Report preparation and project evidence

4.1.1 Report preparation

The report content is grounded in the repository's source files, package declarations, Maven dependencies, application properties and README. Technical claims are limited to behavior visible in those sources. The document does not claim screenshots, performance results or a completed external deployment that were not supplied.

The report follows the five-chapter body structure of the source report and retains its preliminary certificate, contents, figure list, references and ending placeholder page. The contents entries have been aligned with the body headings.

4.1.2 Project management and communication

The system itself supports real-time communication through room topics and participant events. The repository does not include a project management schedule or team communication record; therefore no team roles, meeting cadence or completion dates are asserted.

Testing and data validation

4.1.3 Testing, characterization, interpretation and data validation

The test source contains one Spring Boot context-load test. It mocks the room repository, user repository and messaging template, and uses a test database URI. This test is intended to check that the application context can wire its components without a live MongoDB or broker dependency.

No test execution result, browser test record, load test, latency result or data-integrity measurement was included. Accordingly, observed results are not invented. The implementation behavior can be described from source, while formal validation remains incomplete.

4.1 Implementation results

Area	Implemented behavior	Evidence boundary
Account	Signup/login; BCrypt hash; duplicate email conflict	No JWT or authorization enforcement identified
Rooms	Validated create and lookup by room id	Room id does not prove access rights
Messages	Validate, timestamp, persist, broadcast	No benchmark or delivery guarantee measured
History	Recent-page selection, chronological within page	Stored as embedded room list
Presence	Session mappings and disconnect cleanup	In-memory and single-process
Typing	Transient room broadcast	Not stored

UI	Responsive chat workflow and status feedback	No user study supplied
STOMP routing	Two sessions in one room and a session in another can verify topic isolation	Integration test proposed; not reported as run
Session cleanup	Explicit leave and disconnect paths can verify removal from in-memory participant state	Lifecycle test proposed; not reported as run
History paging	Page zero and subsequent pages can be checked for size and chronological order	Database-backed test proposed; not reported as run

Qualitative outcomes can be organized around each user-visible capability. The account flow supports registration and credential checking; the room flow supports creation and lookup; the conversation flow combines a stored history slice with live room messages; and presence and typing provide temporary activity context. Each capability crosses at least one frontend component and one backend endpoint or event handler, making integration an important part of the observed design.

Message delivery and persistence

Message handling demonstrates the central integration path: the client sends a STOMP payload, the server validates sender and content, resolves the room, creates a timestamped message, saves the room and broadcasts the result. The client appends unique messages to its local view.

History loading demonstrates the persistence path: REST retrieves a bounded page from the room’s message list and the client initializes its display before processing live events. This combination supports continuity across visits, subject to the configured storage being available.

Presence and real-time behavior

Presence handling demonstrates how WebSocket session lifecycle can be reflected in the interface. Join and leave operations update session maps and publish participant data. A disconnect listener removes an abandoned session. Typing events provide a short-lived activity cue without affecting stored history.

The result is a coherent prototype-level feature set. Reliability under network partitions, multiple backend instances and large message collections has not been established by the supplied testing evidence.

Validation should distinguish deterministic source-level checks from environment-dependent behavior. Request validation, status selection and history slicing can be tested with controlled repository responses. MongoDB persistence tests need a running test database or an isolated embedded alternative that behaves like the chosen document store. WebSocket tests need the configured endpoint and broker to be active. Browser-level checks additionally depend on the development server, API base URL and local storage state. Recording these preconditions alongside a test result makes it possible to tell whether a failure indicates an application defect or an unavailable dependency.

For a message sequence, a strong acceptance criterion is that the submitted content is trimmed and validated, the route's room is resolved, the resulting record has a sender, content and server-created time, the room document is saved, and all active subscribers receive the published event. A separate history request should then return the stored item in the expected chronological slice. This criterion checks the full path without relying on subjective performance claims. It should also include an invalid room and a rejected payload, for which no message should be persisted or broadcast. No measured latency or load capacity is stated here because the project materials do not provide benchmark evidence.

4.2 Validation

Validation should check behavior across the client, server, broker and database. The repository contains source validation branches and a context-load test definition, but no completed test matrix. The checks below are proposed; no unrecorded pass results are claimed.

Functional validation should cover both successful and rejected cases. Account tests should verify normalized email, duplicate handling, password hashing and the absence of a password in AuthResponse. Room tests should cover the accepted identifier pattern, duplicate creation, missing lookup and case sensitivity. Message tests should cover blank content, length boundaries, a nonexistent destination room, server timestamp assignment and save-before-broadcast behavior.

Protocol validation should connect more than one session to the same room and a separate session to a different room. Confirm that room topics isolate the event, that join and leave update the participant list and that disconnect cleanup removes stale mappings. Typing events should reach the typing topic but should not be stored as messages. These are proposed execution cases; the project folder does not contain a completed report of their results.

Functional validation

4.2.1 Functional validation

Check account validation, duplicate email, room id constraints, duplicate and missing rooms, blank or oversized messages, persisted history, typing delivery and participant cleanup. These cases are represented in source behavior but were not accompanied by recorded execution results.

Frontend and backend validation

4.2.2 Frontend and backend validation

Exercise route navigation, loading and error states, disconnected send prevention, keyboard behavior, sidebar response, scroll management, HTTP results, STOMP destinations and payload validation. The supplied context test does not cover these cases.

Database validation

4.2.3 Database validation

Confirm unique emails, hashed passwords, room lookup, message persistence and newest-page selection with chronological ordering. The controller clamps history page size to 1-200; no database-backed test results were supplied.

Real-time and security validation

4.2.4 Real-time and security validation

Connect multiple sessions and check room isolation, message delivery, join/leave counts and typing cleanup. Security review must cover authenticated sender identity, room authorization and TLS; the current code does not demonstrate these protections.

Performance validation

4.2.5 Performance validation

Measure connection setup, message latency, throughput at stated session counts, database response time, memory use and history retrieval under a documented workload. No performance measurements were supplied.

4.2.6 Validation summary

The source and context-load test are documented; broader functional, integration, security and performance results remain to be collected.

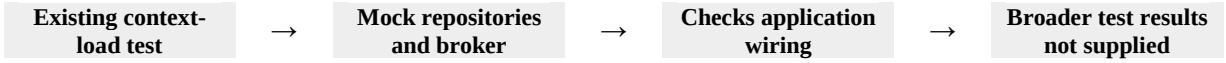


Fig 14. Testing and validation overview

4.3 Verification Strategy and Test Design

4.3.1 Evidence boundary and testing levels

The repository contains a Spring application context test that mocks RoomRepo, UserRepo and SimpMessagingTemplate and supplies a test MongoDB URI. Its test method is named contextLoads and verifies that the application context can be assembled. This is useful wiring evidence, but it does not exercise request validation, MongoDB queries, message mapping behavior, simultaneous clients or React rendering. No broader executed test report is part of the source material reviewed here.

A complete verification plan should separate unit, repository, controller, messaging and browser-level concerns. Unit tests can exercise pure validation and pagination calculations. Repository tests can use a disposable database to verify indexes and document mapping. Controller tests can assert response status and payload shape. Messaging tests can subscribe multiple simulated sessions and confirm topic routing. Browser tests can drive registration, room access, chat and disconnect flows.

Results must distinguish planned tests from completed tests. A test matrix with a blank result field is a plan, not evidence of a pass. For each executed case, record the environment, software versions, initial data, action, expected outcome, actual outcome and date. If a test depends on a local MongoDB service or browser, describe that dependency. This prevents a technical report from turning source-level expectations into unsupported empirical claims.

The current context test mocks core external collaborators so it can check wiring without requiring a live room repository or message broker. That isolation also means the test cannot prove that MongoDB mapping, broker configuration or cross-origin behavior functions against real services. Integration coverage should be added incrementally and should document which external components are real, mocked, embedded or containerized in each run.

4.3.2 Functional and boundary scenarios

Account scenarios should include a valid signup, whitespace trimming, lowercase email normalization, too-short and too-long display names, malformed email, a password below the configured minimum, duplicate email and login with both correct and incorrect credentials. Tests should verify that the response never includes the password hash. A test should also establish the exact behavior of a missing JSON body and of null fields, because controller request binding and explicit null checks can interact.

Room scenarios should verify creation with a valid identifier, identifiers at the length boundaries, invalid punctuation, whitespace around a valid identifier, missing roomId, duplicate creation and lookup of an unknown room. Since the controller supports JSON and plain-text payload forms with a small extractor, tests should cover each supported format and malformed JSON-looking bodies. A uniqueness race should be tested once a database-level unique constraint is introduced.

Message scenarios should verify blank content rejection, whitespace-only content, a one-character message, a message at the maximum accepted size and a message exceeding the maximum. The test should confirm that the room destination controls the stored room, that a nonexistent destination room is rejected, that accepted text and sender are trimmed, and that the resulting event includes a backend timestamp. The payload roomId should be intentionally varied to check the route's authoritative behavior.

History scenarios should cover an empty room, a room with fewer than one page, exactly one page, multiple pages, negative page values, zero or negative sizes, sizes above the clamp and page requests beyond the newest available record. Verify that each returned slice is chronological and that page zero selects the newest items. Because storage is embedded, include a large-list test to characterize the current implementation without claiming that it scales indefinitely.

4.3.3 Real-time, lifecycle and concurrency scenarios

A messaging integration scenario should connect at least two client sessions to one room, subscribe both to the message topic, publish from one session and verify that the server stores and broadcasts the accepted message. A third session in another room should not receive the

event. The scenario should also inspect the persisted room to confirm the event is not delivered if room lookup fails. It should record whether the test broker is in-process or external.
Participant tests should register one session, register a second session with the same display name, register a different name, move a session between rooms, explicitly leave and disconnect unexpectedly. Assertions should reflect the implementation's distinct username list and event count rather than raw session count. After the last session leaves, the room's session set should be removed. Repeated cleanup should be safe and should not create negative counts or stale names.
Typing tests should verify that the room ID in the STOMP path is copied into the broadcast event and that the event does not create a database write. On the client, check that the local user is not shown as typing, that repeated start signals reset the timeout, that a stop signal clears the display and that the receiver removes a stale indicator after its local expiry. These tests should not mistake browser timer behavior for a server guarantee.
Concurrency checks should include simultaneous message submissions, simultaneous joins and a disconnect racing with an explicit leave. The in-memory maps and sets use concurrent collections, but compound operations across several maps are not a single transaction. Tests can expose inconsistent cross-map state and guide a stronger atomic design. Load characterization should specify concurrent sessions, message rate, history size, hardware and database placement; without those details, a latency number is not reproducible.

4.3.4 Frontend, usability and regression scenarios

Frontend tests should verify the landing route, unknown route redirect, guest name validation, room identifier validation, create and join response handling, toast messages and navigation state.
Account tests should check local-storage restoration, successful logout cleanup and the behavior when stored JSON is malformed. Because client storage can be modified, tests should also make clear that these states affect presentation and are not a server authorization mechanism.
Chat page tests should cover the history loading state, empty state, error state, retry action, 404 navigation, successful message rendering, own-message styling, participant updates and typing updates. A race-oriented test can arrange for a live event to arrive while the history request is pending and then confirm that duplicate handling produces one display item. A separate test should document the limitations of the composite deduplication key.
Composer tests should check Enter, Shift+Enter, blank content, maximum length, disabled state while disconnected and send behavior while a publish is pending. Scroll tests should verify automatic following near the bottom and preservation of a reader's position above the bottom. Responsive tests should cover the collapsible sidebar and message area at representative widths. Accessibility checks should include label association, button names, focus order, keyboard use and screen-reader output.
Regression checks should be organized around user-visible contracts rather than implementation snapshots alone. A useful build pipeline would compile the frontend, run component tests and execute backend unit and integration tests. The supplied project has a frontend build script and one backend context test, but no test suite in the inspected folder for all the scenarios described above. The matrix in this section is therefore proposed coverage until those cases are implemented and run.

4.4 End-to-End Communication Scenarios

4.4.1 First connection to a room

The first-connection sequence begins after the landing page has verified the room through REST and navigated to /chat/{roomId}. ChatPage resolves the username, initializes loading state and issues history and participant requests. In parallel, its useStompClient hook creates a STOMP client using the SockJS endpoint. The interface shows a connecting state until the connection callback runs.
On connection, the hook subscribes to message, typing and participant topics for the current room. It stores the returned subscription handles for later cleanup. The hook then publishes a join event containing the sender and room identifier. The server receives the room-specific

destination, obtains the transport session ID and registers the mapping. ParticipantService broadcasts an event based on the distinct current username list.
The client history response seeds the conversation view and a deduplication Set. The participant response seeds a local list that is merged with the current user's name so the display does not depend solely on an event arriving at exactly the same time. A participant event may then update the count and list. The history and socket operations are asynchronous, so a correct client must tolerate either response arriving first.
An implementation review should inspect the loading indicator, the connection badge, the participant sidebar and the empty-history state during this sequence. The important expected behavior is not that every request completes simultaneously, but that temporary partial states remain understandable. A failed history call should not be presented as a connected empty room. A successful history call with an unavailable participant endpoint should still preserve the message content while indicating only what the interface currently knows.

4.4.2 Successful message exchange

A sender composes text and submits it while the connection badge indicates readiness. The frontend checks that the content is nonblank and within the text area's configured maximum, then publishes a JSON MessageRequest to the room-specific application destination. The STOMP client uses its active-connection reference to avoid sending through an inactive client. The UI clears or retains the input according to the send handler's state transitions.
The backend validates content and sender, verifies that the room exists and creates the Message with a server timestamp. It appends the message to the embedded room list and calls RoomRepo.save. After the method returns, Spring routes the result to the topic indicated by @SendTo. All subscribed sessions for the room can then receive the serialized Message. The server uses the destination's room ID to locate and route the data.
The recipient parses the frame as JSON and invokes ChatPage's message callback. That callback computes a deduplication key, ignores the message if it has already been observed and otherwise appends it to state. The message list renders the sender, content and time through MessageBubble. A user whose display name matches the sender sees the own-message presentation. The client also considers whether auto-scroll is enabled before adjusting the viewport.
A complete acceptance trace would record the initial room and database state, the submitted text, the backend log or saved document, the topic event and the resulting screen in both clients. No such trace should be claimed unless actually run. The source flow predicts the outcome under normal operation, but a database outage, dropped connection or concurrent retry can produce a different observation. Reproducible verification should capture both the successful path and a deliberate failure path.

4.4.3 Validation rejection and recovery

A blank message is rejected by server-side StringUtils.hasText even if a modified client bypasses the disabled composer. A message over two thousand characters is rejected. A blank sender is rejected as well. These conditions prevent certain malformed payloads from entering the message list, but they do not establish identity or content moderation. The destination room must also resolve to a Room document before the server appends or broadcasts a message.
At the interface layer, a room identifier that does not match the allowed character rule is caught before the create or join request. A room that passes client validation but fails server validation receives a bad-request response. A duplicate room produces a conflict, while an unknown room produces not found. The landing page maps these outcomes to field messages or toasts instead of assuming that every rejected operation is a network failure.
During history loading, a 404 is treated as evidence that the room cannot be opened and navigation returns to the landing page. Other history failures remain on the chat route with an error state and retry control. A participant-list failure is caught independently and replaced with an empty participant list. This is a deliberate partial-degradation behavior, but it can make the UI's presence view temporarily incomplete.

Failure recovery should avoid losing user input unexpectedly. A future usability test should check what happens to unsent text when a connection drops, whether retry repeats a message, whether the user receives enough information to decide what to do and whether the app avoids announcing success before the backend confirms persistence. The current report does not claim durable offline queuing, automatic retry guarantees or exactly-once delivery.

4.4.4 Leave and unexpected disconnect

An explicit leave begins when the user selects the leave control. The client attempts to publish a leave event for the current room and username, cleans up local subscriptions and deactivates the STOMP client. The backend uses the session ID to remove the associated room and name mapping, removes the session from the room set and broadcasts a participant event when a known mapping existed. The interface then returns to the landing workflow.

A browser close or transport loss may bypass the explicit leave action. Spring emits a SessionDisconnectEvent, and WebSocketEventListener wraps the event message to retrieve the session ID. The listener calls the same participant removal service. This ensures a normal disconnect path can clear a stale map entry, although timing and duplicate lifecycle events should still be covered by tests.

The service removes a room set when its final session is removed. It also avoids broadcasting when its internal removeParticipant call is used as part of moving the same session to another room. The broadcast payload contains a current distinct user list and count. This count can be smaller than the number of sessions when one person uses the same display name in multiple tabs, because the published list is de-duplicated by username.

A reliable lifecycle test should cover explicit leave, unexpected disconnect, double cleanup and the last session leaving a room. It should also verify that removal in one room does not affect another session's membership. Because the service is in-memory, a process crash loses the whole map without publishing a leave event. On restart, connected clients need a reconnect and join sequence to repopulate the state.

4.4.5 History/live overlap and reconnect considerations

The client obtains history through HTTP and live events through STOMP. These channels can overlap: a message may be stored before the history query completes and also arrive through a topic subscription. If the event arrives first, the key enters the seen set before the history response is applied; if history arrives first, the later event should be filtered. The current code initializes seen keys from history and checks the message array before appending a live item.

A possible edge case occurs if history state is replaced after a live event has already updated the message list. The actual effect ordering and React state updates should be tested under controlled delays rather than assumed. A robust reconciliation method can merge both sources by stable message ID and sort by a documented server order. The current composite key is a useful prototype heuristic, but it cannot resolve every collision or ordering ambiguity.

On reconnect, the hook's connection callback can establish subscriptions and send a join notice, while the page may need to refresh history and participants. The inspected code should be evaluated for actual automatic reconnect behavior and state reset behavior before claiming a complete recovery feature. If the library retries transparently, the app still needs to decide how to reconcile events missed during the outage.

A future reconnect contract could include a last-seen message ID, a bounded history refresh and idempotent event merging. It could also make connection health, retry delay and terminal failure visible. Such behavior should be designed around the project's current event schema and storage model. No reconnect protocol or message offset is represented in the existing DTOs, so this remains a design recommendation.

4.5 Detailed Test Case Design

4.5.1 Account and identity cases

Case M-ACC-01 should submit a valid signup with a name, mixed-case email and a password meeting the configured minimum. Verify HTTP 201, a normalized lowercase email in the response, an identifier and name, and the absence of a password field. Verify the repository

receives a User with a BCrypt-formatted password value rather than the literal submitted password. Record the database and test setup so the result is repeatable.
Case M-ACC-02 should vary the display name across blank, one-character, exactly two-character, fifty-character and fifty-one-character values. Confirm the server's lower and upper boundaries and capture the HTTP status and error field. Test leading and trailing spaces to confirm that the server trims before measuring length. A corresponding frontend test should document the guest form's separate maximum of thirty characters.
Case M-ACC-03 should exercise malformed email values, missing email, leading and trailing whitespace, uppercase characters and duplicate normalized addresses. A mixed-case duplicate should still conflict after normalization. If two simultaneous requests attempt the same address, the database unique index should be the final guard; test and document the observed exception mapping rather than assuming the pre-check is race-proof.
Case M-ACC-04 should log in with a correct password, incorrect password, unknown email and blank field. Confirm successful login returns only identity fields and that the two credential failures do not reveal which part was wrong. Separately verify that the client restores local identity after refresh and clears the local keys on logout. Do not treat local-storage restoration as proof that the server authenticated a later chat request.

4.5.2 Room and history cases

Case M-ROOM-01 should create a valid unused room identifier and verify HTTP 201 plus the persisted roomId. Repeat with identifiers at length three and length fifty. Case M-ROOM-02 should test blank values, lengths below three and above fifty, spaces, punctuation, non-ASCII characters and JSON with a missing roomId field. Confirm the error status and that no invalid Room document is saved.
Case M-ROOM-03 should attempt to create an identifier that already exists and verify HTTP 409. Case M-ROOM-04 should query a valid room and an unknown room, verifying HTTP 200 and HTTP 404 respectively. Test exact case differences because repository string matching does not imply case-insensitive room lookup. Include JSON and plain-text creation bodies if both formats are intended to remain supported.
Case M-HIST-01 should test empty history and return an empty array. Case M-HIST-02 should create fewer than fifty messages and verify page zero returns them chronologically. Case M-HIST-03 should create more than one page, verify that page zero returns the most recent segment and page one returns the next older segment, with the expected chronological order inside each response.
Case M-HIST-04 should test page boundaries and size clamping, including a requested size of zero, a negative size and a value above two hundred. Verify that the endpoint clamps rather than returning an invalid slice. A page beyond available history should return an empty array. Repeat with a large embedded history to measure current document retrieval cost without assuming the in-memory slice is a database-side pagination query.

4.5.3 Messaging and event cases

Case M-MSG-01 should submit a valid message to an existing room and verify that content and sender are trimmed, a server timestamp is assigned, the room document is saved and the event is published to the matching room topic. The request's roomId should intentionally differ from the path variable to verify that the destination is used for lookup. This case is about route consistency, not user authorization.
Case M-MSG-02 should reject content that is blank, contains only spaces, exceeds two thousand characters or has a blank sender. Verify that rejected input does not modify the room history.
Case M-MSG-03 should send to a missing room and verify that the operation does not create a new room implicitly. Capture the broker-visible result and any error frame behavior if the test client waits for a response.
Case M-EVT-01 should connect two sessions to one room and one session to another room. Publish a message and verify delivery to both subscribers in the first room and no delivery to the other room. Case M-EVT-02 should send a typing-start event and verify room ID normalization,

topic routing and absence of a MongoDB write. Follow with a typing-stop event and confirm the client removes the indicator.
Case M-EVT-03 should verify deduplication when an identical message is observed through history and a live callback. Case M-EVT-04 should use two different messages with the same sender, timestamp and first forty content characters to expose the limitation of the current composite key. This is a characterization test; if a stable ID is introduced, update the test to assert correct unique rendering by that ID.

4.5.4 Participant lifecycle and concurrency cases

Case M-PRES-01 should add one session and verify the published participant event contains a count of one and the expected username. Add a second distinct session under a different name and verify the sorted distinct list. Add another session with the same name and verify how the event count differs from the raw session count. Assertions should match the current user-list-derived count rather than an assumed unique-account identity.
Case M-PRES-02 should remove one participant explicitly and verify that remaining users stay listed. Remove the final session and verify the room's session set is discarded and its list becomes empty. Repeat removal for the same session to characterize idempotence. Move a session to another room and verify the old room update and new room registration follow the service's no-broadcast-then-add path.
Case M-PRES-03 should trigger SessionDisconnectEvent and verify cleanup without a client leave message. Case M-PRES-04 should race an explicit leave against a disconnect callback and confirm that duplicate cleanup does not create an exception or a negative count. ConcurrentMap and concurrent sets protect individual data structures, but tests should watch cross-map invariants under simultaneous operations.
Case M-PRES-05 should connect and disconnect participants rapidly across several rooms. Record stale membership observations and event ordering. A single-process test can characterize the current implementation, while a multi-instance test should be marked not applicable until shared session state and a distributed broker are added. Do not infer cluster safety from the use of concurrent collections.

4.5.5 Browser workflow and evidence record

Case M-UI-01 should open the landing route, switch between create and join modes, validate guest fields and verify that a valid operation navigates to the room route. Case M-UI-02 should simulate duplicate room, unknown room, server unavailable and malformed input outcomes. Confirm that field messages and toasts are understandable and that loading state clears after both success and failure.
Case M-UI-03 should mount ChatPage with valid route state, with only a stored username, and with neither source available. Verify correct behavior for each condition. Exercise the loading, empty, populated and error states. Verify 404 navigation, retry behavior and participant endpoint failure handling. These cases cover the supported interface branches without assuming a persistent login token exists.
Case M-UI-04 should verify composer keyboard semantics, send-button disabled state, input length, error feedback and scroll behavior. Case M-UI-05 should inspect narrow and wide viewport layouts, sidebar control, focus order and accessible names. A manual checklist should state browser version, viewport, operating system, exact navigation sequence, expected behavior and observed behavior.
For every executed case, store a concise evidence record containing test identifier, build revision, environment, preconditions, input, expected outcome, observed outcome, pass/fail status and any defect reference. Attach logs or screenshots only when they show an actual test run. The cases in this section are proposed checks and are not a claim that all of them have already passed.

4.6 Acceptance Criteria and Requirement Verification

4.6.1 Account and identity acceptance criteria

A registration workflow is acceptable when a user can submit a display name, email and password that satisfy the server's rules, receive a successful response and proceed with the

returned identity fields. The response must not include the password or its stored hash. A duplicate normalized email must be rejected with a conflict response. A malformed email, too-short name or too-short password must be rejected with a clear client-visible error.
A login workflow is acceptable when a valid email and password produce the account identity response and invalid credentials produce the same general unauthorized response regardless of whether the email exists. The UI should not treat a locally cached user object as server authorization. A review should verify that logout clears the client-side identity keys and that an account response with missing fields does not create a misleading authenticated display state.
Identity requirements should explicitly distinguish a guest display name, an account display name, an email address and a trusted authenticated principal. The current chat payload accepts a sender string supplied by the client. Until a signed token or server session is enforced, acceptance criteria for account login must not claim that chat messages are securely bound to the account that logged in.
For each criterion, record the endpoint, validation rule, expected status, expected response properties and test evidence. If one layer provides immediate validation and another layer enforces the same condition on the server, record both. Differences in guest and account name length rules should be either accepted as distinct requirements or reconciled deliberately with corresponding tests.

4.6.2 Room and conversation acceptance criteria

Room creation is acceptable when a valid identifier is trimmed, saved once and returned to the client with a successful creation status. An existing identifier should produce a conflict, and an invalid identifier should be rejected before a room is saved. Room lookup should return the matching room and should distinguish a missing room from a room whose message history is simply empty.
Joining is acceptable when the landing workflow verifies that the room exists, navigates to the room route and obtains an active client connection. The room's participant state should be registered only after the transport session is available. The current participant service treats a WebSocket session as its unit of connection and publishes distinct names; tests should assert the chosen semantics rather than assume that user count and session count are identical.
Message sending is acceptable when a connected client can submit nonblank content within the maximum length, the server validates the destination room, saves a timestamped Message and publishes it to subscribers for that room. A client in another room must not receive the event. A room history request should return the latest page and preserve chronological order within that selected page.
A conversation acceptance record should include both persistence and delivery observations. Check the room document after sending, inspect what each subscribed client renders and reload the page to verify that history is restored. If the result is only a local UI change without a saved record, it does not satisfy the persistence criterion. If no execution was performed, mark the case as planned rather than passed.

4.6.3 Presence and typing acceptance criteria

A join event is acceptable when the server associates the active session ID with the room and display name and publishes an updated participant event. A leave or disconnect is acceptable when the corresponding session mapping is removed and the room's visible participant data is refreshed. If several sessions use the same display name, the current implementation's distinct-name list should be tested and the count semantics documented.
A typing-start event should be delivered only to the room's typing topic and should not be added to the message history. A typing-stop event should remove the sender from the receiving interface. The client should clear an indicator after its local expiration interval if a stop event is lost. The test should confirm these behaviors with at least two sessions and should inspect that no database message record is created by the typing event.
The connection status criterion is met when a user can distinguish at least a connecting, connected, disconnected or error state using a clear visible indicator. The message composer

should not accept a send while it is disconnected. A browser-level check should verify that status changes do not unexpectedly discard the current message text and that the user can understand whether another attempt is needed.
The current implementation is single-process for presence and uses an in-process simple broker. Acceptance criteria for the prototype should not include multi-node presence or broker failover. Those should be introduced as separate future requirements with their own architecture and test evidence, not blended into the current feature checklist.

4.6.4 Evidence classification and sign-off

A requirement may be described as implemented when code paths for it are present. It may be described as verified only when a test or controlled observation exercises the behavior and records the result. It may be described as proposed when it has a design or acceptance criterion but no implementation. This classification prevents source inspection from being presented as empirical performance or acceptance evidence.
A verification record should identify the requirement, software revision, environment, services, test data, exact action, expected result, actual result and evidence location. A failed test should remain in the record with a defect identifier and retest status. A test that could not run because a service was unavailable should be reported as blocked or not executed, not silently omitted.
Academic reporting should avoid invented sign-off. A teacher or reviewer can assess the submitted document, but an official approval is present only when the institution supplies it. The report's certificate and originality placeholder remain institution-dependent. No signature or plagiarism score should be filled in without an authoritative institutional record.
The same discipline applies to deployment and security claims. A local run does not prove public deployment readiness, and password hashing does not prove complete authentication or authorization. Acceptance criteria should state the environment and the protection actually checked. This evidence boundary keeps the project evaluation transparent and makes future improvements measurable.

4.7 Error Handling and Recovery Analysis

4.7.1 Browser-to-server connection failures

An HTTP network failure is caught by roomApi or authApi and converted to a client error with status zero. The interface can then display a message that the backend may not be running. This category differs from an HTTP error response because no server status or response body was received. Troubleshooting should check the configured base URL, backend process, port availability and browser origin before treating the request as a validation defect.
A backend may be reachable over HTTP while the SockJS connection fails. The WebSocket endpoint has its own URL, allowed-origin list and handshake path. A mismatch between VITE_WS_URL and the running server, or a browser origin absent from the configured allowlist, can leave the badge in an error state even when REST calls succeed. Diagnose both channels separately and compare their URLs and logs.
A successful HTTP response does not prove that the database is available for a later operation. Room creation, account lookup and message persistence use repositories. MongoDB may be unavailable at startup or fail during a request. The application should expose a useful server-side diagnostic without leaking internal details to the user, and it should avoid returning success for a write that did not complete.
A reproducible connection-failure test should stop or mock the backend, perform one operation and record the user-facing message and state. A separate origin test should load the frontend from an allowed and a disallowed origin. These tests help distinguish transport reachability from cross-origin policy and server-side validation.

4.7.2 Validation and domain failures

Bad-request responses arise when account fields, room identifiers or message payloads do not satisfy validation. Error messages identify the violated field or rule in many controller branches. The client should preserve input where possible, display the error near the field and allow a

corrected retry. Validation errors should not be logged as unexpected server failures unless they reveal a true defect.

A conflict response from room creation means an identifier already exists. The landing page offers a join-oriented recovery message. A duplicate email conflict during signup suggests using login instead. These cases should remain distinct because repeating the same request is not a useful recovery action; the user must choose a different path or revise the input.

A not-found response for room lookup or history means the logical room identifier does not resolve. The client returns from an unavailable chat route to the landing page or shows an error according to the request path. An empty message array is not the same as not found: it means the room exists but no messages are stored. Keeping these states distinct improves both usability and automated tests.

Message-mapping exceptions may be surfaced through the messaging broker's error mechanism rather than the HTTP error handler. A client should not assume that a rejected STOMP send produces the same JSON error shape as a REST request. An explicit messaging error policy could define error destinations, correlation identifiers and a user-visible explanation for validation, persistence and missing-room failures.

4.7.3 Persistence and partial failures

For messages, the current server method appends to the room list, saves the Room and returns the Message for broadcast. If the repository save fails, the normal successful return path should not occur. Tests should observe this ordering and confirm the UI does not mark an unpersisted message as durable. An explicit acknowledgement would make the sender's state clearer than relying solely on receiving the topic broadcast.

Account signup first checks for an existing email and then saves a User. The unique index provides an additional guard, but a concurrent race may produce a database exception after both requests pass the lookup. A service or exception handler should map that database conflict to the stable HTTP 409 response. Tests should exercise sequential duplicates and, where practical, concurrent attempts.

Room creation also checks for an existing logical identifier before save. A parallel request can race between the check and the write because no logical roomId unique index is shown in the entity. If duplicate rooms would cause ambiguous lookup, a unique constraint should be added and the resulting exception handled. Until then, the pre-check is a helpful validation path but not a concurrency-proof uniqueness guarantee.

The history endpoint reads an entire Room document and creates a sublist. A very large embedded array can increase read and write cost. A partial database failure may affect account access, history or message delivery differently. Operational testing should capture which functions remain available under read failure, write failure and reconnection, rather than treating the application as one binary up/down state.

4.7.4 Client recovery and state consistency

ChatPage has separate loading and error state for history. The retry control reloads the browser, which re-runs the route and its data-loading effect. This is simple, but it may also recreate the socket connection and reset local UI state. A more focused retry could rerun only the failed request, though that would require careful coordination with current subscriptions and deduplication state.

The participant request is intentionally caught separately from the history request. If participants are unavailable but history succeeds, the UI can still show the conversation. This is a useful partial-failure choice, but the interface should avoid implying that an empty participant list is confirmed when it may be merely unavailable. A loading or stale indicator could make this state clearer in future work.

On a WebSocket disconnect, the hook updates connection status, cleans subscriptions and exposes disconnected state. Sending is guarded by the current connected reference. The project should be tested for whether input remains available after a disconnect and how the user recovers.

Automatic retries, queueing unsent messages and exactly-once resubmission are not claimed as existing behavior here.
State reconciliation should be deterministic. If history, live events and participant snapshots arrive in a different order, the page should converge to a coherent view. Stable identifiers and a reducer that processes ordered event actions could simplify reasoning. Until then, controlled delayed-response tests are a practical way to expose timing defects without assuming that local development network latency represents every deployment.

4.7.5 Diagnostic logging and user messages

The application.properties file sets INFO logging for Spring WebSocket and DEBUG logging for the application package. This is useful in development but should be reviewed for sensitive data before deployment. Logs should avoid passwords, bearer tokens and raw private message content unless a justified diagnostic policy permits them and safeguards retention and access.
A user-facing message should explain the next safe action without exposing a stack trace, internal file path or database URI. Developer diagnostics can include a correlation identifier and server-side cause. The current client displays response error text when available, so server error messages should remain concise and should not contain confidential configuration values.
A consistent diagnostic record could include timestamp, severity, operation type, room identifier where appropriate, session correlation and a sanitized error category. Identifiers can still be personal or sensitive in some deployments, so logs require access and retention rules. Metrics should count categories and durations rather than copying full message content into telemetry.
Troubleshooting documentation should map symptom to likely layer, evidence to inspect and a safe correction. For example, a failed REST request calls for base URL and backend checks; a failed SockJS handshake calls for endpoint and origin checks; a missing room response calls for identifier and database checks. Clear separation reduces random configuration changes that can hide the underlying cause.

4.8 Performance Evaluation Plan

4.8.1 Define a reproducible workload

Performance evaluation should start by defining the workload. State how many rooms are active, how many sessions join each room, how frequently users send messages, how many characters each message contains and how long each test runs. Include the mix of idle and active users, whether history is being loaded and whether typing events occur. Without these details, two reported latency values may describe entirely different conditions.
The environment record should identify CPU, memory, operating system, Java version, Node version, MongoDB version and whether services run on the same machine. Record the backend port, frontend origin, database URI category and network topology without exposing passwords. Warm-up duration, measurement duration and number of repetitions should be stated so startup effects are not confused with steady-state behavior.
A useful baseline can start with one room and a small number of browser sessions, then increase concurrent sessions and message rate in controlled steps. Each step should use the same message size distribution and history depth. The aim is to observe how the current single-process broker, concurrent session maps and embedded message documents behave, not to claim a production capacity number from a short demonstration.
Test data should be generated in a repeatable way and removed or isolated between runs. Use a dedicated test database rather than a personal or production database. Record whether the room document already contains messages, because document size affects read and write costs. Preserve the workload script or manual procedure alongside raw results so another evaluator can reproduce the run.

4.8.2 Select metrics and instrumentation

Connection setup time can be measured from initiating the SockJS/STOMP client until the connected callback. Record failures and timeouts separately from successful durations. Message round-trip time requires a timestamp or correlation mechanism that links a send to a received event; the current Message has a server timestamp but no client acknowledgement or unique ID,

so measurement instrumentation may need a test-only correlation value or a carefully defined observation method.

HTTP metrics can include request duration and status counts for signup, login, room creation, room lookup, history and participant reads. Database metrics can include operation duration, document size and query behavior. WebSocket metrics can include active sessions, connect/disconnect rate and published events. Client metrics can record rendering delay and scroll responsiveness. Avoid logging full message content as a measurement shortcut.

Latency summaries should report a distribution such as median and a high percentile, not only an average. Throughput should include successful messages per second and failure rate under a stated workload. Resource use should be measured at the same time so a faster response is not achieved through unbounded memory growth. If the test is too small to produce stable percentiles, report the sample count and the exploratory nature of the observation.

The current embedded history model should be measured at several document sizes. Compare history request duration and message-send save duration as a room's list grows. This characterizes the present approach and can inform a migration threshold. The experiment should not extrapolate beyond tested sizes or imply that one local machine represents a production cluster.

4.8.3 Establish thresholds and interpret results

Acceptance thresholds should be chosen before observing results. Examples include a maximum tolerable connection time under the test environment, an acceptable error percentage for a fixed message rate, or a target history response time for a specified message count. Thresholds should be tied to user needs and the prototype's scope rather than selected after seeing a favorable measurement.

If a threshold is missed, investigate the relevant layer using traces or logs. A slow message operation could reflect document serialization, MongoDB write time, broker delivery, browser rendering or network delay. A controlled test should isolate one factor at a time where possible. Changing the data model and load level simultaneously makes it difficult to attribute the result.

Interpretation should state limitations. A local simple broker and a local database do not characterize a remote deployment. A handful of browser windows does not establish concurrent-user capacity. Results from one laptop can be affected by background tasks and thermal conditions. A report should state whether the workload was synthetic or human-generated and whether the test ran once or was repeated.

A useful comparison could evaluate the existing embedded-room model against a proposed separate-message collection in a branch or controlled prototype. Keep the same message set, request sizes and hardware. Compare read and write behavior, but account for implementation differences and indexes. The comparison remains exploratory unless the method and code revision are preserved and repeated.

4.8.4 Concurrency and stability evaluation

Concurrent-user evaluation should include simultaneous connections, joins, messages and disconnects. Observe whether participant events remain coherent, whether all intended room subscribers receive messages and whether one room's traffic leaks into another. Increasing session count gradually can reveal a threshold where connection setup, message latency or browser rendering degrades.

Soak testing can run a modest workload for a longer period while recording memory, open connections, error rates and message history growth. The embedded message array and in-memory session maps have different lifecycle behavior: messages accumulate in MongoDB, while session maps should shrink as users disconnect. The test can check for stale sessions and unbounded browser state without assuming the application has a cleanup policy for persisted content.

Reconnect and interruption experiments should stop the backend or database briefly, restore it and record what clients observe. Check whether the status badge transitions, whether the user can re-enter the room, whether a history refresh restores saved messages and whether missed

participant state is rebuilt. The current project should not be credited with automatic recovery beyond what is actually observed in such a run.

A stability result should include the duration, workload, restarts, failure events and recovery observations. If no failure occurred during a short run, say only that none was observed under those conditions. Do not claim fault tolerance or production reliability from a clean local session. The code and configuration should be archived with the result so a later release can be compared fairly.

4.8.5 Report performance evidence responsibly

A final performance table should separate environment, workload, metric, sample count, result summary and interpretation. Include the tested code revision and date. Use units consistently and state how the metric was computed. If a measurement was not captured, leave it as not measured rather than inserting an estimate based on general expectations.

Avoid converting a local observation into a universal claim. A measured average under one room, a few sessions and a small history describes only that configuration. It does not show how the application behaves under many rooms, a remote database, packet loss or a different browser. The report should identify conditions near each number so the reader can interpret its scope.

If a benchmark shows the current storage design becoming slower as room history grows, connect the finding to a proposed architectural response such as separate message records and indexed pagination. If the connection layer is the bottleneck, evaluate broker configuration and client lifecycle. Recommendations should follow measured evidence, not be stated as completed optimization work before implementation.

No performance measurements were provided in the project materials reviewed for this report. this section defines a reproducible plan for future evaluation and does not assert benchmark outcomes. Once tests are executed, replace the plan status with dated evidence, preserve the raw observations and revise conclusions only where supported by those results.

CHAPTER 5
CONCLUSION AND FUTURE WORK

The implementation demonstrates a coherent minimum real-time conversation workflow across its browser, service and persistence layers. React routes and page components collect account, room and message input; REST controllers implement account, room and history operations; a STOMP handler accepts room events; MongoDB stores account and room documents; and an in-memory service tracks active participants. The key engineering outcome is the integration of snapshot-oriented HTTP operations with a subscription-oriented event channel. The distinction between persisted messages and transient typing or presence is an important part of that design.

The implementation should be evaluated within its verified boundary. Account creation hashes passwords, but later chat events are not tied to a verified principal. Room identifiers are locators and do not enforce membership. Messages are embedded in a growing room document and lack a stable unique message identifier. Presence and the simple broker are local to one server process. The source contains a context-load test, not a comprehensive test suite or measured performance study. These limitations do not negate the educational value of the working structure; they identify the engineering work required before the application could make stronger claims about security, scale, delivery guarantees or operational readiness.

5.1. Conclusion

The Multi-user Chatroom Application demonstrates a room-based communication workflow using a React browser client, a Spring Boot backend, STOMP over SockJS and MongoDB. Users can register or log in, create or join rooms, load recent messages, send real-time messages and see participant and typing updates.

The implementation separates REST operations from ongoing messaging and separates durable records from transient session state. Message content is validated, timestamped, stored with the room and broadcast to subscribers. Participant sessions are tracked concurrently in memory and removed on disconnect. The interface presents connection status and handles common loading and input states.

The project demonstrates integration of a browser interface, HTTP APIs, a broker-based event channel and a document database in one room-oriented workflow. It makes the distinction between finite REST operations and ongoing message delivery visible in the code. It also separates durable room history from transient typing and participant state. These are useful design outcomes for understanding how a small real-time application can be decomposed.

Project contribution and implementation limits

The project provides a useful foundation for demonstrating full-stack integration and event-driven application design. Its current boundaries are important: the repository does not show token issuance, room authorization, distributed broker configuration, production deployment, end-to-end encryption or comprehensive automated test results.

The application should therefore be understood as a prototype, not a production-secure messaging service. Further engineering should prioritize authenticated identity, access control, reliable persistence scaling, automated tests and deployment configuration.

A practical implementation roadmap should sequence these improvements so that each step has a verifiable outcome. First, define a message contract with a server-generated identifier, validated room reference, authenticated sender and timestamp, then cover its serialization with API and broker tests. Second, introduce authorization checks that derive the sender from the authenticated connection and confirm the requested room membership before accepting a message or presence event. Third, separate message persistence from the Room aggregate and provide indexed, bounded history queries with stable ordering. A migration should preserve existing room messages and validate their counts before the old representation is retired. Fourth, move ephemeral presence and broker routing to shared infrastructure only if a multi-instance deployment is an actual requirement. Each stage should include failure cases, such as expired credentials, inaccessible rooms, database outages and reconnects, so that the user interface can communicate the state accurately.

The roadmap also provides a basis for measurable evaluation. A test environment can record message acceptance, persistence completion and delivery to each subscribed client, then compare those outcomes under a stated number of concurrent sessions and message rates. Reliability checks should include a backend restart, a dropped network connection and recovery from the history endpoint. Security review should examine password handling, origin configuration, transport protection, authorization boundaries and accidental exposure in logs. Usability evaluation can observe whether participants understand the active room, connection indicator and transient typing cue. These measurements must be conducted and documented before they are presented as results. The source project currently establishes a functional educational prototype and provides a narrow context-load test; it does not supply such load, security or user-study evidence. Accordingly, the roadmap is a proposal for responsible extension, not a claim about completed work.

Interface and API evolution should be planned as a coordinated contract change. The existing browser obtains history through a REST request and then subscribes to the room's STOMP topic; if the server changes a message field or timestamp format, both paths must continue to produce records that the same rendering component understands. A versioned response contract, explicit date-time format and shared examples would make this expectation reviewable. The client should also distinguish a stale history snapshot from a live event already received during loading, using a stable message identifier to reconcile the two sources. Its current composite duplicate key uses sender, time and a content prefix; this reduces some overlap but cannot prove identity when two messages share those values. Introducing a server-generated ID would allow reliable reconciliation, editing, deletion and moderation audit trails if those features were later designed.

Room lifecycle requires similar care. The current create operation establishes a room document, while entering a room checks its existence and message submission resolves it again. This is a simple lifecycle that avoids implicit creation during posting, but it does not define ownership, expiry, invitation or deletion. Future room controls should specify who can create, join, rename

or close a room and what happens to retained messages when a room is removed. These decisions affect data retention and user expectations, so they should precede interface controls. Participant presence currently represents active WebSocket sessions rather than durable membership; a participant list should therefore not be presented as an access-control list. The distinction between being allowed to enter and being connected at this moment is fundamental to an accurate multi-user experience.

Transient event behavior also benefits from explicit policy. Typing notifications are intentionally short-lived and do not belong in message history. The client sends activity as the user types and stops after inactivity; receiving clients clear indicators after a timeout. A future implementation could suppress redundant frames, handle tab suspension and disconnection explicitly, and ensure that a user's own typing event is not shown back to that same user. Such optimizations should preserve the principle that typing is advisory: it must never block message delivery or be treated as evidence that a message was sent. Participant events, in contrast, should be reconciled after reconnect so that the UI can recover a fresh snapshot rather than relying on missed incremental changes. These details are suitable future refinements, not additional behavior claimed for the current code.

5.2. Future Work

Future improvements should strengthen identity and access control, scale message storage and broker delivery, and expand automated validation. Each enhancement below is proposed work; it is not represented as already implemented.

5.2.1. Secure authentication and room authorization

Issue a signed session or token after login, validate the authenticated principal on REST and STOMP requests, and enforce room membership before reading history or publishing messages. Add expiry, revocation, CSRF/origin protections as applicable, and TLS for deployed environments.

A staged roadmap should begin with requirements and acceptance cases for authenticated identity, room membership and message ownership. The backend can then issue a signed credential or establish a server session, validate it on REST and STOMP paths, and derive sender identity from the verified principal. Room access checks should apply consistently to history, participant reads and message publication. Tests should include forged names and cross-room access attempts.

Scalable message storage

5.2.2. Scalable message persistence

Move messages to a dedicated collection with room and timestamp indexes, stable message identifiers and database-side pagination. Define retention and archival rules to prevent unbounded room documents and preserve efficient history retrieval.

5.2.3. Distributed messaging and presence

Replace the in-process simple broker with a broker relay or distributed messaging service if multiple backend instances are required. Coordinate presence across instances and define reconnect, duplicate session and heartbeat expiry behavior.

Message delivery and moderation

5.2.4. Stronger message identity and delivery

Assign each message a unique server-generated identifier and consider acknowledgements, idempotency keys, ordering policy and client retry behavior. This would improve deduplication and make delivery status more precise during reconnection.

5.2.5. Moderation and room controls

Add room ownership, invitation or access policy, participant removal, reporting and moderation tools. These features require authenticated identity and a clear policy model before they can be safely implemented.

Automated testing and operations

5.2.6. Expanded automated tests

Add unit tests for input validation and paging; controller tests for HTTP responses; repository integration tests; STOMP integration tests with multiple sessions; and frontend tests for key user workflows. Record the environment and actual results for each test.

5.2.7. Observability and operations

Add structured logs, health checks, metrics and alerts for connection failures, database errors and message processing. Provide environment-based configuration, backup/restore instructions, dependency update practices and deployment documentation.

Accessibility and rich interactions

5.2.8. User experience and accessibility

Conduct usability testing and improve keyboard navigation, focus handling, screen-reader labels, contrast and responsive behavior. Consider virtualized rendering for large message histories and a clearer retry experience after disconnection.

5.2.9. Rich communication features

Potential additions include message editing/deletion, attachments, reactions, search and notifications. Each would require explicit persistence, validation, authorization, storage and moderation decisions.

Privacy and data retention

5.2.10. Privacy and retention

Document what account and message data is stored, for how long and who can access it. Define deletion and export processes, secure backups, encryption at rest where appropriate, and a privacy notice aligned with the deployment context.

5.2.11. Deployment and scaling

Prepare separate development and production profiles, container or managed-service deployment, secure secret storage, TLS termination, database availability planning and load testing. Publish performance results only after controlled measurement.

Deployment and next steps

With these improvements, the application could grow from a learning prototype into a more robust collaborative messaging service. The next development phase should begin with identity and authorization, because later moderation and room privacy depend on trustworthy user identity.

Subsequent work should add tests alongside each feature and document acceptance criteria before claiming completion. Scaling decisions should be guided by measured usage rather than assumed requirements.

5.3 Security, Reliability and Scalability Review

5.3.1 Identity, trust boundaries and access control

The account endpoints validate registration fields, normalize email and hash passwords using BCrypt. These are positive foundations for account handling, but they do not complete an authorization system. The login response contains the user identifier, name and email and does not issue a signed access token or session cookie. AuthContext keeps this response in local storage for client presentation. Local storage can be changed by a browser user and must not be treated as a trusted credential by the backend.

MessageRequest includes a sender value supplied by the client. ChatController checks that this value is not blank, then stores it as the sender. The controller does not derive the sender from an authenticated principal. A participant can therefore claim another display name in the current protocol. Likewise, a room ID functions as a destination label; the inspected handlers do not enforce a membership or ownership policy before returning room data or accepting messages.

A security extension should first define the identity lifecycle: how a user proves credentials, how a server-issued session is carried over HTTP and STOMP, how credentials expire, and how logout or revocation behaves. The authenticated identity should be associated with the messaging session during connection and then used for sender attribution. Room membership should be checked on both history retrieval and message publication rather than inferred from a client-provided room ID.

Transport security is another boundary. The local development setup uses localhost addresses and does not establish a production HTTPS/WSS deployment. A real deployment should use TLS, a restrictive origin policy, safe cookie or token handling, rate limits and monitoring. These are proposed safeguards, not functionality represented as completed in the current source. A threat review should include impersonation, room enumeration, message flooding, stored content abuse and credential reuse.

5.3.2 Reliability and failure handling

A message path crosses several components: browser input, STOMP client, SockJS transport, Spring message mapping, room lookup, MongoDB save and topic broadcast. Failure at any stage can change what the sender and recipients observe. The current interface disables sending until the connection is established and surfaces connection state, but it does not provide a durable delivery acknowledgement tied to a unique message ID.

If a browser disconnects after publishing but before observing a broadcast, the user may not know whether the server stored the message. Retrying could create a duplicate because the server does not accept an idempotency key. The current frontend's composite key can reduce duplicate display when the same sender, timestamp and text prefix recur, but it does not guarantee deduplication across reconnects or concurrent writes. Explicit acknowledgements and stable identifiers would make this state easier to resolve.

The history endpoint is a recovery mechanism for accepted messages because it reads the persisted room document on a later visit. It does not repair all live-event gaps while the page remains open. Reconnection handling should resubscribe, refresh participant state, reload a history window and merge incoming events without losing scroll position. Such a process needs a clear ordering rule, especially if the message list changes while a history request is still in flight.

MongoDB availability is required for account and message persistence. If the database becomes unavailable during a message send, persistence may fail before the broadcast return. The application should return a comprehensible error and avoid implying a saved message when it could not store one. Future error handling should distinguish validation rejection, missing room, database failure, broker failure and client transport failure rather than collapsing all errors into a generic notice.

5.3.3 Capacity, persistence growth and multi-instance operation

Participant tracking is memory resident and scoped to a backend process. A restart clears it, which is acceptable for current presence state but requires clients to reconnect and register again. With two backend instances, a session connected to one instance is not automatically visible to

the other instance's participant endpoint or in-memory maps. Sticky routing alone would not provide shared broadcast state if room subscriptions are split across instances.
The simple broker is in-process. It supports local topic routing in a single application instance but does not act as a durable distributed broker. Scaling the backend horizontally would require a broker relay or another shared messaging service, plus a strategy for user destinations, connection lifecycle, heartbeat behavior and participant state. The operational design should be tested with clients distributed across instances before claiming horizontal scaling.
Embedded room messages make the document grow as a conversation grows. The history controller slices the loaded list after obtaining the room; it does not execute a database-side paginated query over independent message records. A room with an unbounded message array can eventually exceed practical document size and update cost. A dedicated message collection, compound room/timestamp index and retention policy would address these concerns more directly.
Capacity tests should define a workload instead of presenting a single unexplained number. Relevant variables include simultaneous sessions, rooms, message size, event frequency, history depth, database location and network latency. Measurements could include connection establishment, message round-trip time, database write duration, memory per session and history response time. No such benchmark is supplied with the current report, so the existing project is described as a prototype rather than a measured high-capacity service.

5.3.4 Operational governance and privacy

The application stores account fields and message content in MongoDB but does not provide a data-retention policy in the source. Before deployment, the owner should decide how long accounts, rooms and messages remain, who can request deletion and how backups are handled. These choices affect the embedded room model because a message is nested within its room document rather than stored as an independently addressed record.
Logs are configured for application and WebSocket categories, including a DEBUG level for the application package. This can help development diagnosis but should be reviewed before production because verbose logs may expose identifiers or implementation details. A deployed service should define which fields are safe to log, how long logs are retained, how operational errors are alerted and how log access is controlled.
Operational readiness also requires health checks for the HTTP process and database connection, backup and restore procedures, dependency update practices and a controlled release process. The inspected repository does not demonstrate a container deployment, cloud configuration or automated release pipeline. These should be treated as future operational work rather than inferred from the existence of a Vite build or Maven wrapper.
Privacy notices should explain that message history is persisted and identify the retention and access rules once those rules exist. User display names and account email addresses have different purposes; the interface currently shows the account email in the sidebar when available, while messages use a display name. A privacy review should check where each value appears, how it is stored and whether data minimization or deletion requirements apply to the deployment context.

5.4 Maintainability and Extension Design

5.4.1 Responsibility boundaries and refactoring opportunities

The frontend already separates pages, reusable components, API helpers, context and hooks. That separation provides natural points for incremental changes. A future feature should first identify whether it belongs to route-level orchestration, shared state, transport handling, presentation or validation. Keeping fetch details in API modules and STOMP lifecycle details in hooks reduces the risk that every component implements its own network behavior.
ChatPage still coordinates many concerns, including history, participants, message deduplication, typing state, scrolling, input state and responsive sidebar controls. It can remain readable if each state transition is described and tested, but adding reactions, search and older-history paging

would make the component more difficult to maintain. Candidate extractions include useRoomHistory, useParticipants, useMessageTimeline and a reducer for ordered room events.
On the backend, AuthController currently combines request validation, normalization, duplicate lookup, hashing, persistence and response construction. A service boundary would make the account policy reusable and allow focused unit tests independent of HTTP status mapping. Room creation and message persistence could likewise be placed behind services that coordinate validation, repository operations and domain outcomes.
Refactoring should preserve observable contracts. Before moving logic, capture tests for status codes, response shapes, validation boundaries, message ordering and event destinations. Then move one responsibility at a time and run the relevant tests. This incremental approach avoids mixing behavioral redesign with package restructuring and makes regressions easier to locate. The existing project is small enough for such changes to remain comprehensible if contracts are documented.

5.4.2 Stable identifiers and event contracts

The Message entity currently carries sender, content and time but no unique message identifier. The browser derives a display key from sender, timestamp and content prefix. Adding a server-generated ID would support reliable rendering keys, acknowledgement tracking, idempotent retries and future edit/delete operations. A migration plan would be required for existing embedded messages because older records do not carry that ID.
ParticipantEvent provides roomId, count, type, username and users. Its event type is represented as a string with values such as JOIN and LEAVE. An enum or schema validation could make these values explicit. The client currently accepts event fields conditionally and updates its participant list based on the event's full users list or on a single username and type. A versioned event contract would reduce ambiguity as the system evolves.
TypingEvent contains sender, roomId and a boolean. Its lightweight shape fits a transient activity event. Future additions such as a correlation ID, event timestamp or expiry should be driven by a concrete client behavior, not added without need. A protocol document should specify required fields, optional fields, destination patterns, sender authority, error behavior and compatibility policy for all event types.
Timestamps should be treated consistently. Persisted messages receive a server-side LocalDateTime, while clients use it in deduplication and rendering. A more portable contract would use an explicit offset or UTC instant and a stable serialized format. This would reduce ambiguity across time zones and make cross-service ordering easier to reason about. The current code does not implement a global sequence number.

5.4.3 Feature extension sequence

An extension sequence should begin with identity enforcement because room privacy, moderation and trustworthy sender attribution depend on it. Define how the backend validates an authenticated principal for REST and STOMP operations, how room membership is granted and revoked, and how an unauthorized request is represented. Add tests for forged senders and attempts to read or publish to rooms outside the user's membership.
The next foundation is stable message identity and independently paginated storage. Move embedded messages to a message collection when data volume warrants it, add room and timestamp indexes, and define ordering and retention. Preserve an API response shape or provide a versioned migration so existing clients can continue to render messages. Validate migration behavior against rooms with empty, small and large histories.
After the data and identity foundations, room ownership, invitations, moderation and message deletion can be considered. Attachments and rich media require storage limits, content-type validation, malware scanning, authorization and retention rules. Search requires an indexing approach and privacy policy. These features create new abuse and operational surfaces, so they should be planned with explicit requirements rather than treated as simple UI additions.
Finally, improve multi-instance operations only when usage needs justify them. A shared broker, shared presence store, reconnect protocol, metrics and load tests should be designed together.

Each phase should have acceptance criteria and measured outcomes. This sequence reduces the chance of building scale mechanisms around an unverified workload while still preparing a clear path from a local academic prototype to a more robust service.

5.5 Usability, Accessibility and Responsible Use

5.5.1 Labels, keyboard use and focus

The landing page uses labels associated with input identifiers and displays validation messages near relevant fields. Evaluation should confirm that a screen reader announces the field label, its error state and the error text when the user moves focus. The auth card, room field and display-name field should expose clear labels rather than relying only on placeholders. Placeholder text can disappear during typing and is not a substitute for a persistent label.

The chat composer provides an explicit button label for sending and uses a keyboard convention where Enter sends while Shift+Enter adds a line break. This convention should be verified with keyboard-only use and communicated to assistive technology users. Ensure that keyboard focus remains visible, that disabled state is conveyed and that the send action can be triggered without a pointer.

The sidebar menu and close controls should have meaningful accessible names and a predictable focus sequence. Opening the sidebar should not strand focus behind an overlay, and closing it should return focus to the control that opened it. The leave-room control is an important navigation action and should be distinguishable from merely closing the sidebar.

Testing should use both automated accessibility checks and manual assistive technology review. Automated checks can identify missing labels and some contrast issues, but cannot determine whether a dynamic participant update is announced helpfully or whether the conversation reading order is sensible. Record the browser, screen reader and interaction sequence used for any accessibility claim.

5.5.2 Dynamic updates and status communication

The connection badge changes as the STOMP client connects, disconnects or encounters an error. A visual-only color cue may not convey the state to a person with color-vision differences or to a screen reader. The label should include readable text or an accessible status role. Evaluate whether frequent reconnect changes are announced once, without overwhelming a user with repetitive messages.

The participant count and list can change as join and leave events arrive. The interface should communicate the updated information without shifting keyboard focus. A polite live region can announce meaningful changes, while a full list update may be too verbose. The desired behavior should be established with user testing rather than inferred from the presence of a React state update.

Typing indicators are transient and may change repeatedly. Their accessibility behavior should avoid announcing each keystroke or flapping state. The existing hook throttles sender events and the client expires stale indicators after a timeout, which offers a starting point for a restrained announcement policy. If the indicator is decorative for assistive technology, that choice should be deliberate and documented.

Loading, empty and error states should each have clear text and appropriate semantics. A retry button should be reachable and labeled. If history loading completes while focus is in another control, content should not unexpectedly steal focus. Validation errors should remain associated with their inputs, and toast notifications should be announced in a way that does not interrupt composing a message.

5.5.3 Readability, responsive layout and motion

Message bubbles should remain readable at browser zoom and narrow viewport widths. Evaluate long URLs, long words, multiple lines and messages near the maximum length. Text should wrap without widening the page or hiding the sender and timestamp context. The collapsible sidebar should preserve access to the participant list and connection state on small screens.

The interface uses icons from Lucide React and decorative assets in the landing-page preview. Icons that communicate actions need accessible labels, while purely decorative graphics should

be hidden from assistive technology. The chat preview on the landing page should not be mistaken for live content. Visual separation can help users understand the transition from demonstration preview to active room conversation.

Typing dots and other transitions may involve motion. Users who prefer reduced motion should not be forced to view continuous animation. CSS should be checked for reduced-motion preferences and for transitions that do not delay an important state change. The interface's information should remain available when animation is disabled.

Responsive evaluation should include both portrait and landscape layouts, browser zoom and virtual keyboard behavior on a mobile device. The composer must remain visible when the soft keyboard opens, and scroll-to-bottom behavior should not conceal the last message behind the composer. A screenshot at one desktop size is not sufficient evidence of responsive behavior across devices.

5.5.4 Usability study protocol

A small usability study could ask participants to create a room, invite a second browser session, exchange a message, locate a previous message and leave the room. Participants should receive neutral task instructions and should not be coached through hidden interface steps. The study should record task completion, observed confusion and participant feedback without capturing passwords or unnecessary personal data.

Study metrics might include completion, time to task, input corrections, help requests and perceived clarity. These should be reported with the number and characteristics of participants and the test environment. A small formative study can identify obvious friction but should not be presented as statistically representative or as proof of broad usability.

Privacy safeguards should include consent, a clear explanation of recorded observations and a retention plan for notes or recordings. Account creation can be tested with mock or disposable credentials where possible. If real messages are used, participants should know that the application stores messages in MongoDB and that the prototype does not claim production-grade access control.

Findings should lead to traceable improvements. Each observation can be connected to a proposed change, a responsible component and a follow-up verification case. For example, confusion about a missing room can lead to revised error text in LandingPage and a regression test for the not-found response. No study outcomes are asserted in this report because no participant study evidence was supplied.

5.5.5 Content moderation and respectful communication

The current chat prototype provides room-based text exchange but does not implement reporting, blocking, moderation or administrator controls. If the application is opened to a wider audience, the project owner should decide how users can respond to abusive content and who may remove messages or participants. These policy decisions affect both the backend authorization model and the interface.

A moderation design should distinguish a room-level removal action from deleting a user's account and from hiding content locally. It should record which identity performed an action, what target was affected, when it occurred and whether an appeal or reversal is possible. Such an audit trail should be protected from ordinary message authors and should have a defined retention period.

Rate limiting and spam controls are relevant even for a small service because message endpoints can be flooded. A server should enforce limits independently of the browser composer. Content filtering alone is not a complete moderation strategy; human reporting and escalation routes may be necessary depending on deployment context.

Any future moderation feature should be accompanied by user-facing policy language and tested for both ordinary and adversarial cases. The current report treats moderation as future work and does not claim that messages are screened, encrypted, deleted on request or governed by a completed community policy.

REFERENCES

1. Spring. Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/index.html
2. Spring. Spring Framework WebSocket Support. https://docs.spring.io/spring-framework/reference/web/websocket.html
3. Spring. Spring Data MongoDB Reference. https://docs.spring.io/spring-data/mongodb/reference/
4. Spring. Spring Security Crypto Module. https://docs.spring.io/spring-security/reference/features/integrations/cryptography.html
5. React. React Documentation. https://react.dev/learn
6. Vite. Vite Guide. https://vite.dev/guide/
7. React Router. Documentation. https://reactrouter.com/
8. STOMP.js. STOMP over WebSocket client documentation. https://stomp-js.github.io/
9. SockJS. SockJS client documentation. https://github.com/sockjs/sockjs-client
10. MongoDB. MongoDB Manual. https://www.mongodb.com/docs/manual/
11. Oracle. Java Documentation. https://docs.oracle.com/en/java/
12. Apache Maven. Maven Wrapper documentation. https://maven.apache.org/wrapper/
13. MDN Web Docs. WebSocket API. https://developer.mozilla.org/en-US/docs/Web/API/WebSocket
14. W3C. The WebSocket API. https://websockets.spec.whatwg.org/
15. Lucide. Lucide documentation. https://lucide.dev/
16. Oracle. Java Platform, Standard Edition 21 API Specification. https://docs.oracle.com/en/java/javase/21/docs/api/
17. Spring. Spring Framework Web MVC Reference. https://docs.spring.io/spring-framework/reference/web/webmvc.html
18. Spring. STOMP over WebSocket Messaging. https://docs.spring.io/spring-framework/reference/web/websocket/stomp.html
19. World Wide Web Consortium. STOMP Protocol Specification, Version 1.2. https://stomp.github.io/stomp-specification-1.2.html
20. IETF. RFC 6455: The WebSocket Protocol. https://www.rfc-editor.org/rfc/rfc6455
21. OWASP Foundation. Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
22. OWASP Foundation. Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
23. MongoDB. Embedded Data Models. https://www.mongodb.com/docs/manual/data-modeling/embedding/
24. Spring. Spring Boot Testing Reference. https://docs.spring.io/spring-boot/reference/testing/index.html
25. React Router. Routing Documentation. https://reactrouter.com/start/declarative/routing
26. W3C. Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/
27. Mozilla. Using the WebSocket API. https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

Project-specific implementation claims are based on the supplied source files: pom.xml, package.json, application.properties, Java controllers/entities/configuration/services, React components, hooks and API modules.

Plagiarism Report

The supplied source report ends with a plagiarism report page, but no originality report or similarity result was provided with the project files. No score, certificate, approval or institutional finding has been fabricated.

Attach the institution-issued originality report on this page before submission. This placeholder is editable and preserves the final-page position of the supplied report.

Institution-issued report to be inserted here.